

OpenAI API Deep Dive



Authentication • Request Structure • Streaming

Week 3 • Module 1: Foundations & LLM Essentials • Section 3.1

Nunnari Academy

Generative AI & Agentic AI Program

Three Core Areas



API Authentication

Secure key management, environment variables, and .env configuration



Request Structure

Chat Completions API, message roles, parameters, and response objects



Streaming Responses

Real-time token delivery, chunk handling, and production patterns

API Authentication

An abstract network diagram consisting of a series of interconnected nodes and lines, overlaid on a background of scattered dots. The nodes are represented by circles of varying sizes, and the lines are thin, connecting some of the nodes in a path that roughly follows the shape of the text 'API Authentication'.



API Authentication

A unique secret token issued by OpenAI that identifies and authorizes your application to access their services.

Correct Way to Use Your Key

```
# ❌ WRONG — Never hardcode keys
client = OpenAI(api_key="sk-proj-abc...")

# ✅ CORRECT — Use environment variables
import os
from openai import OpenAI
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
```

Storing Securely with .env

```
# 1. Create .env file
OPENAI_API_KEY=sk-proj-your-key-here

# 2. Add .env to .gitignore
echo ".env" >> .gitignore

# 3. Load in Python
from dotenv import load_dotenv
load_dotenv()
client = OpenAI()
# Auto-reads from env
```

Key Management Rules

⚠️ **Never commit to GitHub**
Bots scan public repos within seconds

⚠️ **Rotate keys periodically**
Limits damage if compromised

⚠️ **Separate key per project**
Revoke one without breaking others

⚠️ **Set usage limits**
Prevents unexpected billing surprises

Getting Your API Key

1. Sign up at platform.openai.com
2. Dashboard → API Keys → Create new
3. Copy immediately — shown only once

Request Structure

The background is a solid purple color. It features a decorative pattern of small, light purple dots arranged in a grid. Overlaid on this is a line graph consisting of several large, dark purple circular nodes connected by straight lines. The graph starts at the bottom left, moves up to the right, then down to the right, then up to the right, and finally down to the right, ending at the bottom right.



Anatomy of an API Call

Required: model + messages. Everything else (temperature, max_tokens, penalties) is optional.

```
from openai import OpenAI
client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o",           # Which model
    messages=[
        {"role": "system",
         "content": "You are a helpful
                    assistant."},
        {"role": "user",
         "content": "Explain transformers
                    in 2 sentences."}
    ],
    temperature=0.7,          # Creativity
    max_tokens=500,           # Max length
    top_p=1.0,                # Vocab diversity
)

print(response.choices[0].message.content)
```

Key Parameters

Param	Range	Use When
temperature	0.0-2.0	Low=factual High=creative
max_tokens	1-128K	Control cost & length
top_p	0.0-1.0	Alt to temp (don't combine)
freq_penalty	-2.0-2.0	Reduce word repetition
pres_penalty	-2.0-2.0	Push new topics

finish_reason Values

stop: ✓ Finished naturally

length: ⚠ Hit max_tokens

content_filter: ⛔ Safety filtered

Message Roles & Multi-Turn Conversations

system

Sets the LLM behavior and persona

```
"You are a concise  
Python tutor."
```

user

The human input or question

```
"What is a decorator?"
```

assistant

LLM's previous reply
— simulates memory

```
Used to maintain  
conversation history
```

Multi-Turn: You Manage History Manually

```
messages = [  
    {"role": "system", "content": "You are a coding tutor."},  
    {"role": "user", "content": "What is a for loop?"},  
    {"role": "assistant", "content": "A for loop repeats..."},  
    {"role": "user", "content": "Show me an example."}  
]
```

Key Rule

The **assistant** role is how you simulate memory.

Without it, every API call starts fresh with zero context.

The Response Object

```
{
  "id": "chatcmpl-abc123",
  "model": "gpt-4o-2024-08-06",
  "choices": [{
    "message": {
      "role": "assistant",
      "content": "A transformer is..."
    },
    "finish_reason": "stop"
  }],
  "usage": {
    "prompt_tokens": 45,
    "completion_tokens": 120,
    "total_tokens": 165
  }
}
```

How to Read It

Your Answer

```
response.choices[0]
  .message.content
```

Why It Stopped

```
response.choices[0]
  .finish_reason
```

Token Usage

```
response.usage
  .total_tokens
```

Always check `finish_reason`
"`length`" = truncated output!

Streaming Responses

The background is a solid purple color. It features a decorative pattern of small, dark purple dots scattered across the surface. A network of larger, dark purple dots is connected by thin, dark purple lines, forming a complex, interconnected shape. A line graph is also visible, consisting of a series of dark purple dots connected by dark purple lines, showing a fluctuating trend. The text "Streaming Responses" is written in a white, sans-serif font, positioned on the left side of the image.



Streaming Responses

Normal vs Streaming

Without Streaming: [..... 3 second wait] → Full response arrives at once

With Streaming: [token][token][token][token]... → Words appear instantly

Enabling Streaming — Just Add stream=True

```
stream = client.chat.completions.create(  
    model="gpt-4o",  
    messages=[  
        {"role": "system", "content": "Helpful assistant"},  
        {"role": "user", "content": "Explain neural nets"}  
    ],  
    max_tokens=500,  
    stream=True # ← This one flag!  
)  
  
for chunk in stream:  
    token = chunk.choices[0].delta.content  
    if token is not None:  
        print(token, end="", flush=True)
```

Key Difference

Normal:

response.choices[0].message.content

Streaming:

chunk.choices[0].delta.content

delta = "what's new in this chunk"
— only the newly generated token(s)

How Chunks Arrive

Chunk 1: "Neural"
Chunk 2: " networks"
Chunk 3: " learn"
Last: finish="stop"

Production Streaming Patterns

Collect Full Response While Streaming

```
full_response = ""

for chunk in stream:
    token = chunk.choices[0].delta.content
    if token is not None:
        print(token, end="", flush=True) # Stream
        full_response += token           # Collect

# Use full_response for history/DB/logging
messages.append(

    {"role": "assistant", "content": full_response}
)
```

With Error Handling

```
try:
    for chunk in stream:
        if finish == "length":
            print("\n ⚠ Truncated")
        elif finish == "content_filter":
            print("\n ⛔ Blocked")
except openai.RateLimitError:
    print("Rate limit - retry")
```

When to Use

Situation	Mode
Chat interfaces	⚡ Stream
Long-form content	⚡ Stream
Batch processing	⏸ Normal
Short responses	⏸ Normal
Audit / logging	⏸ Normal
Need token counts	+options

Token Usage in Streaming

```
stream_options=

{"include_usage": True}

# Usage arrives in final chunk
if chunk.usage: # print stats
```

Section Summary

The background is a solid purple color. It features a decorative pattern of small, light purple dots arranged in a grid. Overlaid on this is a line graph with a darker purple line connecting several circular nodes of varying sizes. The nodes are positioned at various points across the slide, with some being part of the line graph and others standing alone.

Section Summary



Authentication: API key goes in .env, loaded via `os.getenv()` — never hardcoded. Rotate keys, set billing limits, use one key per project.



Request Structure: Needs model + messages at minimum. Messages carry role + content. All other parameters are optional tuning knobs.



Message Roles: system sets behavior, user asks, assistant holds history — you manage conversation state manually every request.



Response Object: Answer lives at `response.choices[0].message.content`. Always check `finish_reason` — "length" means truncated output.



Streaming: Add `stream=True`. Access tokens via `chunk.choices[0].delta.content`. Collect into a string for history. Use for all user-facing chat.

Next → 3.1 continued: Function Calling & Structured Outputs

Function Calling & Structured Outputs



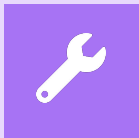
Function Calling • *Structured Outputs*

Week 3 • Module 1: Foundations & LLM Essentials • Section 3.1

Nunnari Academy

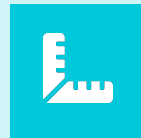
Generative AI & Agentic AI Program

Two Core Areas



Function Calling

Describe tools to the model.
It decides when to call them
and what arguments to pass.
Your code executes the function.



Structured Outputs

Guarantee the model's response
always matches your schema.
Valid JSON, correct types,
every field present.

Function Calling

The background is a solid purple color. It features a decorative pattern of small, dark purple dots scattered across the surface. A series of larger, dark purple dots are connected by a solid purple line, forming a zigzag path that starts from the left and extends towards the right, passing behind the text.



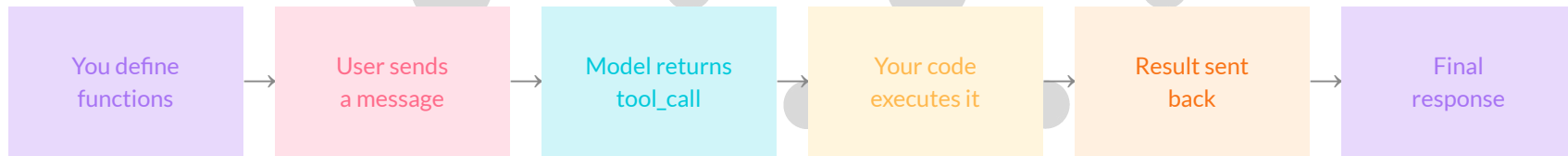
What is Function Calling?

Function calling lets you describe custom tools to the model. The model decides when to call them and what arguments to pass — your code executes the actual function.

Why It Matters

This is the foundation of AI agents. Instead of guessing or hallucinating, the model can trigger real actions — query a database, call an API, look up live data, run a calculation.

How It Works — The Full Flow



Step 1 & 2 – Define Function + Tool Schema

Step 1 – Python Function

```
def get_weather(city: str, unit: str = 'celsius') -> dict:
    # In reality, this would call a weather API
    return {
        'city': city,
        'temperature': 28,
        'unit': unit,
        'condition': 'Sunny'
    }
```

Write Good Descriptions

The model uses your description field to decide WHEN to call the function. Be specific – treat it like a prompt.

Step 2 – Tool Schema

```
tools = [{
    'type': 'function',
    'function': {
        'name': 'get_weather',
        'description': 'Get the current weather
                        for a given city,',
        'parameters': {
            'type': 'object',
            'properties': {
                'city': {
                    'type': 'string',
                    'description': 'City name, e.g. Chennai'
                },
                'unit': {
                    'type': 'string',
                    'enum': ['celsius', 'fahrenheit']
                }
            },
            'required': ['city']
        }
    }
}]
```

Steps 3 & 4 – Send Request + Check Response

Step 3 – Send the Request

```
from openai import OpenAI
import json

client = OpenAI()

messages = [
    {'role': 'user',
     'content': 'What is the weather like
                 in Chennai right now?'}
]

response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=messages,
    tools=tools,
    tool_choice='auto'
)
```

Step 4 – Check for Tool Call

```
response_message = response.choices[0].message

# Check if model decided to call a tool
if response_message.tool_calls:
    tool_call = response_message.tool_calls[0]

    print(tool_call.function.name)
    # → 'get_weather'

    print(tool_call.function.arguments)
    # → '{"city": "Chennai",
    #   "unit": "celsius"}'

    # Parse the arguments
    args = json.loads(
        tool_call.function.arguments
    )
```

Steps 5 & 6 — Execute & Send Result Back

```
# Execute your actual function
result = get_weather(**args)

# Add the assistant tool_call message to history
messages.append(response_message)

# Add the tool result as a new message
messages.append({
    'role': 'tool',
    'tool_call_id': tool_call.id, # Must match the tool_call id
    'content': json.dumps(result)
})

# Send everything back for the final response
final_response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=messages,
    tools=tools
)

print(final_response.choices[0].message.content)
# → 'The weather in Chennai is currently 28°C and sunny.'
```

tool_choice Options

Value	Behavior
"auto"	Model decides to call or respond directly
"none"	Never calls functions — text only
"required"	Must call at least one function
{name: "..."} (in orange)	Force a specific named function

Multiple Tools & Parallel Calls

Multiple Tools — Model Picks the Right One

```
tools = [  
  {  
    'type': 'function',  
    'function': {  
      'name': 'get_weather',  
      'description': 'Get current weather',  
      'parameters': { ... }  
    }  
  },  
  {  
    'type': 'function',  
    'function': {  
      'name': 'search_flights',  
      'description': 'Search for flights  
        between two cities',
```

```
      'parameters': {  
        'type': 'object',  
        'properties': {  
          'from_city': {'type': 'string'},  
          'to_city': {'type': 'string'},  
          'date': {'type': 'string'}  
        },  
        'required': ['from_city', 'to_city', 'date']  
      }  
    }  
  }  
]
```

Multiple Tools & Parallel Calls

Parallel Tool Calls

```
# 'Weather in Chennai AND Mumbai?'  
# Model returns TWO tool_calls  
  
response = client.chat.completions.create(  
    model='gpt-4o-mini',  
    messages=messages,  
    tools=tools,  
    tool_choice='auto'  
)  
  
if response_message.tool_calls:  
    for tool_call in response_message.tool_calls:  
        args = json.loads(  
            tool_call.function.arguments)  
        result = get_weather(**args)
```

```
messages.append({  
    'role': 'tool',  
    'tool_call_id': tool_call.id,  
    'content': json.dumps(result)  
})  
  
# Send all results back at once  
final = client.chat.completions.create(  
    model='gpt-4o-mini',  
    messages=messages, tools=tools  
)
```

Structured Outputs

The background is a solid teal color. It features a pattern of small, dark teal dots scattered across the surface. A prominent graphic element is a line graph composed of larger dark teal dots connected by straight lines. This graph starts on the left, rises to a peak, falls to a trough, and then rises again towards the right. The text 'Structured Outputs' is written in a white, sans-serif font, with 'Structured' on the top line and 'Outputs' on the bottom line, positioned on the left side of the image.



What are Structured Outputs?

Structured Outputs guarantee that the model's response always matches a schema you define — valid JSON, every field present, correct data types.

Two Approaches

Function Calling

Model decides WHEN to call a function and what arguments to pass

Structured Outputs

Model ALWAYS returns a guaranteed JSON shape — no function involved

Two Methods for Structured Outputs

Method 1 — JSON Mode

Simple, less strict. Guarantees valid JSON but does NOT enforce a specific schema.

Method 2 — Pydantic (Strict)

Most powerful. Define exact schema using Pydantic — guaranteed every time.

Method 1 – JSON Mode

Guarantees valid JSON output but does not enforce a specific schema. Good for quick extraction tasks.

```
response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[
        {'role': 'system',
         'content': 'You are a data extractor. Always respond in valid JSON.'},
        {'role': 'user',
         'content': 'Extract: John Doe, 28 years old, Software Engineer at Infosys'}
    ],
    response_format={'type': 'json_object'} # ← Ensures valid JSON output
)

import json
data = json.loads(response.choices[0].message.content)
print(data)
# → {'name': 'John Doe', 'age': 28, 'role': 'Software Engineer', 'company': 'Infosys'}
```

Limitation

JSON Mode guarantees **VALID JSON** but does **NOT** guarantee a specific schema — fields can still be missing or have wrong data types.

Best For

Quick extraction tasks, prototyping, or situations where you can tolerate minor schema variations.

Method 2 – Structured Outputs with Pydantic

The most powerful method. Define your exact schema using Pydantic and OpenAI guarantees the response matches it – every time, every field, every type.

```
# Step 1: Define your exact schema
class PersonProfile(BaseModel):
    name: str
    age: int
    role: str
    company: str
    email: Optional[str] = None

# Step 2: Use parse() instead of create()
response = client.beta.chat.completions.parse(
    model='gpt-4o-mini',
    messages=[
        {'role': 'system', 'content': 'Extract structured information.'},
        {'role': 'user', 'content': 'John Doe, 28, Software Engineer
                                     at Infosys. Email: john@infosys.com'}
    ],
    response_format=PersonProfile
)
```

Key Advantage

No manual JSON parsing needed. Access fields as typed Python attributes directly from the parsed object.

Guarantees

- Correct data types (int, str, etc.)
- All required fields present
- No hallucinated keys
- Optional fields handled

Model Support

All examples use gpt-4o-mini – full support at a fraction of gpt-4o cost.

Nested Schemas — Complex Structured Data

Structured Outputs handles deeply nested objects and lists without any extra parsing logic.

```
class LineItem(BaseModel):
    product: str
    quantity: int
    price: float

class Order(BaseModel):
    order_id: str
    customer: str
    items: List[LineItem] # ← Nested list of objects
    total: float
    currency: str

response = client.beta.chat.completions.parse(
    model='gpt-4o-mini',
```

Zero Parsing Logic

Pydantic validates and casts every field automatically — nested lists, floats, ints, all typed correctly.

```
messages=[
    {'role': 'system', 'content': 'Extract order details as structured data.'},
    {'role': 'user', 'content': '''
        Order #1042 for Priya Sharma:
        - 2x Laptop Stand at 1500 each
        - 1x USB Hub at 800
        Total: 3800 INR
    '''}
],
response_format=Order
)
```

Structured Outputs + Function Calling

Combine both — define a function AND enforce a strict output schema on its arguments using `strict=True`.

```
class WeatherArgs(BaseModel):
    city: str
    unit: str = 'celsius'

tools = [{
    'type': 'function',
    'function': {
        'name': 'get_weather',
        'description': 'Get current weather for a city',
        'parameters': WeatherArgs.model_json_schema(), # ← Pydantic schema as
    }
}]

tool_params = {
    'strict': True # ← Enforce strict schema
}

on arguments
}
}]

response = client.chat.completions.create(
    model='gpt-4o-mini',
    messages=[{'role': 'user', 'content': 'Weather in Coimbatore?'}],
    tools=tools,
    tool_choice='auto'
)
```

What `strict=True` Does

Guarantees the function arguments match your Pydantic schema exactly — correct types, required fields, no extras.

Combined Power

Function calling for ACTIONS (run code, call APIs).

Structured outputs for EXTRACTION (pull data from text).

Together → production-ready LLMs.

JSON Mode vs Structured Outputs

Feature	JSON Mode	Pydantic
Guarantees valid JSON	✓	✓
Guarantees specific fields	✗	✓
Guarantees correct types	✗	✓
Handles nested objects	✗	✓
Requires Pydantic	✗	✓
Best for	Quick extraction, prototyping	Production, complex schemas

Real-World Use Cases

Resume Parser

Invoice Processor

Sentiment Analyzer

Product Catalog

Support Triage

Meeting Summarizer

Section Summary

The background is a solid purple color. It features a decorative pattern of small, light purple dots arranged in a grid. Overlaid on this is a line graph with a darker purple line connecting several circular nodes of varying sizes. The nodes are positioned at various points across the slide, with some larger nodes acting as focal points for the line segments.

Section Summary



Function Calling: Describe functions to the model using a tool schema. The model decides when to call them and passes structured arguments. Your code executes the function and returns the result. This is the foundation of all AI agents.



Structured Outputs: Guarantee a fixed JSON schema in every response. Use JSON Mode for quick extraction; use Pydantic + `.parse()` for production — it enforces types, required fields, and nested structures with zero parsing errors.



Model Choice: All examples use gpt-4o-mini — it fully supports function calling, JSON Mode, and Structured Outputs at a fraction of the cost of gpt-4o. Ideal for both learning and production workloads.



Combined Power: Use function calling for ACTIONS (run code, call APIs). Use structured outputs for EXTRACTION (pull structured data from unstructured text). Together, they make LLMs truly production-ready.

Vision Capabilities with GPT-4V

Image Understanding • *Multimodal Reasoning* • *Visual AI*

Week 3 • Module 1: Foundations & LLM Essentials • Section 3.1

Nunnari Academy

Generative AI & Agentic AI Program

Four Core Areas



Sending Images

URL & Base64 methods,
supported formats,
content array structure



Detail Levels

Low vs High resolution,
token cost implications,
auto mode selection



Practical Use Cases

Chart analysis, OCR,
UI-to-code, multi-image,
multi-turn conversations



Cost & Limitations

Token estimation,
pricing model, and
what GPT-4V cannot do

What is
GPT-4V?

The background is a solid purple color. It features a decorative pattern of small, semi-transparent purple dots scattered across the surface. A line graph, also in a semi-transparent purple, is overlaid on the dots. The graph consists of several circular nodes connected by straight lines, forming a jagged, upward-trending path that starts from the left and moves towards the right side of the image.



What is GPT-4V?

GPT-4V (Vision) is OpenAI's multimodal model that can see and understand images alongside text. Send a photo, screenshot, diagram, chart, or document scan – and the model reasons about it just like text.

Why It Matters

Most real-world data is not just text. Invoices are PDFs. Reports have charts. Products have photos. GPT-4V lets your app understand all of it – without separate CV pipelines.

Supported Formats

image/png

image/jpeg

image/gif

image/webp

What GPT-4V Can Do

Image Description

Object Detection

Text Extraction (OCR)

Chart Analysis

Document Understanding

Visual Q&A

Code from Screenshots

Image Comparison

Sending Images





Method 1 – Image URL (Simplest)

Use when your image is already hosted online and publicly accessible.

```
from openai import OpenAI
client = OpenAI()

response = client.chat.completions.create(
    model='gpt-4-vision-preview',
    messages=[{
        'role': 'user',
        'content': [
            {'type': 'image_url',
             'image_url': {
                 'url': 'https://example.com/image.png'
             }},
            {'type': 'text',
             'text': 'Describe what you see in this image.'}
        ]
    }],
    max_tokens=500
)

print(response.choices[0].message.content)
```

Key Structure

Images go inside the content array as type: 'image_url'. Combine with type: 'text' for your prompt.

Important

The image URL must be publicly accessible. Private URLs or localhost will NOT work.

Model Name

Use gpt-4-vision-preview for all vision tasks. Handles text + image natively.



Method 2 — Base64 Encoded Image

Use when you have a local image file that is not hosted anywhere.

```
import base64
from openai import OpenAI
client = OpenAI()

def encode_image(path: str) -> str:
    with open(path, 'rb') as f:
        return base64.b64encode(
            f.read()).decode('utf-8')

base64_img = encode_image('invoice.png')

response = client.chat.completions.create(
    model='gpt-4-vision-preview',
    messages=[{
        'role': 'user',
        'content': [
            {'type': 'image_url',
             'image_url': {'url':
                f'data:image/png;base64,{base64_img}'
            }},
            {'type': 'text',
             'text': 'Extract line items and totals.'}
        ]
    }], max_tokens=800
)
```

Data URI Format

data:image/png;base64,{encoded_data}

The prefix tells the API the image format.

Best For

Local files, user uploads, screenshots, pipeline outputs — anything without a public URL.

Size Limit

Maximum 20MB per image. Larger images must be resized before encoding.

URL vs Base64 – When to Use Which

	URL Method	Base64 Method
Image location	Hosted online (public URL)	Local file or private storage
Setup	Just paste the URL	Read file → encode → send
Request size	Small (URL string only)	Large (full image in payload)
Privacy	Image must be public	Stays local until sent
Best for	Quick testing, hosted assets	User uploads, private data

Setting the Detail Parameter

```
'image_url': {  
  'url': 'https://example.com/image.jpg',  
  'detail': 'high'    # ← 'low' | 'high' | 'auto' (default)  
}
```

Detail Controls Cost

'low' = ~85 tokens flat.
'high' = 1,000–2,000+ tokens.
Choose wisely!

Image Detail Levels

The background features a smooth gradient from light pink at the top to white at the bottom. A network of dark red dots and lines is scattered across the upper half, resembling a molecular or neural network structure. A large, solid white circle is positioned on the right side, partially overlapping the gradient and the network.



Image Detail Levels

GPT-4V processes images at different resolutions depending on the detail parameter. This directly affects cost and accuracy.

Detail	How It Works	Tokens	Best For
"low"	Resizes to 512×512, single tile	~85	Quick classification, simple questions
"high"	Full resolution, multiple tiles	1,000–2,000+	Charts, text, fine details, documents
"auto"	Model decides based on image size	Varies	General use (default)

Cost Tip

Use 'low' for simple yes/no or classification tasks. Use 'high' only when fine detail matters — reading small text or analyzing complex charts. 'high' can cost 20x more than 'low'.

Token Calculation Formula

```
# Low detail: Always ~85 tokens flat
# High detail: 85 base + 170 per 512x512 tile
# (image split into tiles for full resolution)
```



Vision Token Cost & Estimator

High Detail Cost by Image Size

Image Size	Tiles	≈ Tokens
512 × 512	1	255
1024 × 1024	4	765
2048 × 2048	16	2,805
4096 × 4096	16*	2,805*

* Scaled to 2048×2048 max before tiling

Key Insight

Images >2048px are always scaled down first. A 4096px image costs the same as a 2048px image in 'high' mode.

Quick Token Estimator

```
def estimate_vision_tokens(width, height):  
    # Scale to fit within 2048x2048  
    scale = min(2048/width, 2048/height, 1.0)  
    w = int(width * scale)  
    h = int(height * scale)  
  
    # Count 512x512 tiles  
    tiles = ((w+511)//512) * ((h+511)//512)  
    return 85 + (170 * tiles)  
  
print(estimate_vision_tokens(1920, 1080))  
# → ~595 tokens  
  
print(estimate_vision_tokens(800, 600))  
# → ~595 tokens  
  
print(estimate_vision_tokens(512, 512))  
# → ~255 tokens
```

Practical Use Cases

The background is a solid orange color. It features a network of small orange dots scattered across the surface. A prominent line graph is overlaid, consisting of larger orange dots connected by solid orange lines. The graph starts on the left, rises to a peak, falls to a trough, and then rises again towards the right. The text 'Practical Use Cases' is written in a white, sans-serif font on the left side of the image.

01 Chart & Graph Analysis

```
response = client.chat.completions.create(  
    model='gpt-4-vision-preview',  
    messages=[  
        {'role': 'system',  
         'content': 'You are a data analyst. Analyze charts  
                     and provide clear business insights.'},  
        {'role': 'user',  
         'content': [  
             {'type': 'image_url',  
              'image_url': {'  
                  'url': 'https://example.com/sales_chart.png',  
                  'detail': 'high'  
              }},  
             {'type': 'text',  
              'text': 'What trend does this chart show?  
                     What are the top 3 insights?'}  
         ]},  
    ],  
    max_tokens=600  
)
```

System Prompt Sets Role

Use a system prompt to make the model a domain-specific analyst — it provides more structured, professional insights.

High Detail Required

Charts need 'high' detail to read axis labels, legends, data points, and small text accurately.

Output Options

Combine with Structured Outputs to get typed data (trends, percentages, recommendations) instead of free text.

Invoice Extraction + Structured Outputs

```
class LineItem(BaseModel):
    description: str
    quantity: int
    unit_price: float
    total: float

class Invoice(BaseModel):
    vendor: str
    invoice_date: str
    invoice_no: str
    items: List[LineItem]
    subtotal: float
    tax: float
    grand_total: float

img = encode_image('invoice.png')
resp = client.beta.chat.completions.parse(
    model='gpt-4-vision-preview',
    messages=[{'role': 'user', 'content': [
        {'type': 'image_url', 'image_url': {
            'url': f'data:image/png;base64,{img}',
            'detail': 'high'}},
        {'type': 'text',
         'text': 'Extract all invoice details.'}
    ]}], response_format=Invoice)
```

Vision + Pydantic

Combine GPT-4V with Structured Outputs for reliable, typed data extraction from document images — zero manual parsing.

Accessing the Parsed Data

```
invoice = response.choices[0]
            .message.parsed

print(f'Vendor: {invoice.vendor}')
print(f'Total: {invoice.grand_total}')

for item in invoice.items:
    print(f' - {item.description}: '
          f'{item.total}')
```

Production Pattern

Vision for input + Structured Outputs for guaranteed schema — the standard production pattern.

03

Screenshot to Code (UI Mockup)

```
base64_image = encode_image('ui_mockup.png')

response = client.chat.completions.create(
    model='gpt-4-vision-preview',
    messages=[
        {'role': 'system',
         'content': 'You are a frontend developer.
                    Convert UI designs into clean HTML
                    and Tailwind CSS.'},
        {'role': 'user',
         'content': [
             {'type': 'image_url', 'image_url': {
                 'url': f'data:image/png;base64,
                     {base64_image}',
                 'detail': 'high'}},
             {'type': 'text',
              'text': 'Convert this UI mockup into
                      HTML with Tailwind CSS.'}
         ]},
        max_tokens=2000)

html_code = response.choices[0].message.content
```

Design → Code

Send a UI mockup or screenshot and get production-ready HTML + CSS. Ideal for rapid prototyping.

System Prompt Matters

Specify the framework (Tailwind, Bootstrap), responsiveness, and coding style in the system prompt.

High Detail + max_tokens

Use 'high' detail to read UI text and set max_tokens=2000+ for complete HTML output.

Multi-Image & Multi-Turn Vision

04 Multiple Images in One Request

```
img1 = encode_image('product_before.jpg')
img2 = encode_image('product_after.jpg')

response = client.chat.completions.create(
    model='gpt-4-vision-preview',
    messages=[{
        'role': 'user',
        'content': [
            {'type': 'text',
             'text': 'Compare these images. Differences?'},
            {'type': 'image_url', 'image_url': {'url':
                f'data:image/jpeg;base64,{img1}',
                'detail': 'low'}},
            {'type': 'image_url', 'image_url': {'url':
                f'data:image/jpeg;base64,{img2}',
                'detail': 'low'}}
        ]},
    ],
    max_tokens=500)
```

05 Multi-Turn Conversation

```
# Turn 1 – Send image + question
messages = [{ 'role': 'user',
               'content': [
                   {'type': 'image_url', 'image_url': {'url':
                       f'data:image/png;base64,{base64_image}',
                       'detail': 'high'}},
                   {'type': 'text',
                    'text': 'What is shown in this dashboard?'}
               ]}]

resp1 = client.chat.completions.create(
    model='gpt-4-vision-preview',
    messages=messages, max_tokens=500)
messages.append({'role': 'assistant',
                 'content': resp1.choices[0].message.content})

# Turn 2 – Follow-up (no image resend!)
messages.append({'role': 'user',
                 'content': 'Which metric dropped most?'})

resp2 = client.chat.completions.create(
    model='gpt-4-vision-preview',
    messages=messages, max_tokens=300)
```

Key Rule: Once the image is in conversation history, the model remembers it for all follow-up turns. No resending needed.

Limitations & What GPT-4V Cannot Do

The background features a gradient from light pink at the top to a darker red at the bottom. A network of dark red dots of varying sizes is connected by thin lines, creating a molecular or network-like structure. A prominent line graph with several data points is also visible, showing an upward trend.



Limitations & What GPT-4V Cannot Do

No image generation

GPT-4V is input-only. Use DALL·E 3 for image generation.

20MB image size limit

Larger images must be resized or compressed before sending.

Struggles with tiny text

Very small text in dense documents may be misread. Use 'high' detail and crop when possible.

No spatial coordinates

Cannot return bounding boxes or pixel coordinates for detected objects.

No real-time / video

Cannot process video streams or live camera feeds — single frames only.

Privacy sensitivity

Be cautious with sensitive images — data is processed by OpenAI's servers.

Handwriting unreliable

Printed text works well; cursive and messy handwriting often fails.

Section Summary

The background is a solid purple color. It features a decorative pattern of small, dark purple dots arranged in a grid. Overlaid on this grid is a line graph consisting of larger, dark purple circular nodes connected by straight lines. The graph starts at a low point on the left, rises to a peak, falls to a trough, and then rises again towards the right. The text 'Section Summary' is written in a large, white, sans-serif font on the left side of the image.

Section Summary



GPT-4V: Multimodal model (gpt-4-vision-preview) that understands images and text together. No separate CV pipeline needed — it reasons about photos, diagrams, charts, and scanned documents natively.



Two Input Methods: URL for hosted images; Base64 for local files. Both are sent as image_url type inside the content array of a user message.



Detail Levels: Use 'low' for simple tasks (~85 tokens flat). Use 'high' for charts, documents, fine text (1,000–2,000+ tokens). Never pay for high detail when low is sufficient.



Power Combinations: Vision + Structured Outputs = reliable typed data extraction. Vision + multi-turn = image stays in history, no resend needed. Vision + System Prompt = consistent analysis.



Key Limitations: Input only (no generation), no bounding boxes, struggles with handwriting, 20MB image size limit. Use DALL·E 3 separately if image generation is needed.

Audio & Speech APIs

Whisper (STT) & TTS



Speech Recognition • *Voice Synthesis* • *Voice Pipelines*

Week 3 • Module 1: Foundations & LLM Essentials • Section 3.1

Nunnari Academy

Generative AI & Agentic AI Program



Overview – OpenAI's Audio Stack

Two dedicated audio APIs form a complete voice pipeline – from transcription to speech synthesis.

Whisper (STT)

Audio → Text

Transcribes or translates spoken audio into text. Supports 99 languages.

TTS

Text → Audio

Converts written text into natural-sounding speech with 6 voices.

Voice Pipeline

End-to-End

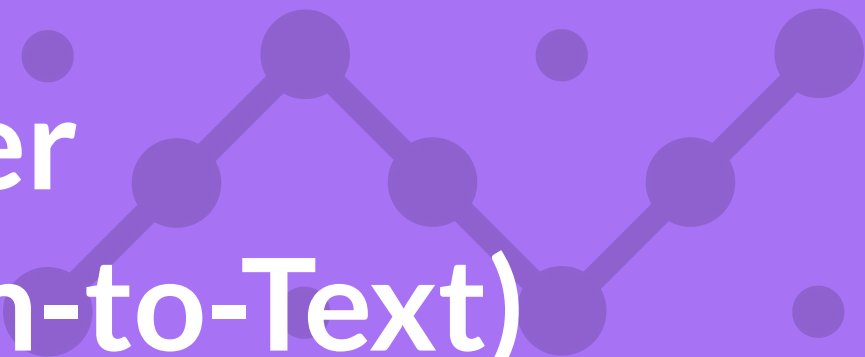
Whisper + GPT + TTS = complete voice-in, voice-out AI assistant.

Audio is not a niche feature – it is increasingly the primary interface for AI products.

Part 1

Whisper

(Speech-to-Text)





What is Whisper?

OpenAI's open-source ASR model, trained on 680,000 hours of multilingual audio. Handles accents, noise, technical jargon, and code-switching with remarkable accuracy.

Key Strengths

Multilingual

Supports 99 languages

Noise-Robust

Handles background noise

Accent-Tolerant

Works across accents

Technical Vocab

Recognizes domain terms

Long-Form Audio

Handles long recordings

Zero Setup

No training needed



Transcription vs Translation

Mode	Endpoint	Input	Output
Transcription	/audio/transcriptions	Audio in any language	Text in the SAME language
Translation	/audio/translations	Audio in any language	Text ALWAYS in English

Translation mode is a one-step pipeline – it transcribes AND translates to English in a single API call. No separate step needed.

Supported Audio Formats

mp3

mp4

mpeg

mpga

m4a

wav

webm

ogg

flac

Whisper Pricing

\$0.006 per minute of audio, billed per second. A 1-hour meeting costs approximately \$0.36 to transcribe.

Language Hint Tip

Providing the language code speeds up transcription by 10–20% and reduces errors. Always set it when you know the source.



Basic Transcription

```
from openai import OpenAI
client = OpenAI()

with open('meeting_recording.mp3', 'rb') as f:
    transcript = client.audio.transcriptions.create(
        model='whisper-1',
        file=f
    )

print(transcript.text)
# → 'Good morning everyone. Today we
#    will discuss Q3 targets...'
```

Model

Use 'whisper-1' — it's the only available model. OpenAI handles version updates server-side.

File Input

Open the file in binary mode ('rb'). The API accepts the file object directly — no base64 encoding needed.

Output

Returns a Transcription object with .text attribute containing the full transcribed text.

Use the prompt parameter to maintain spelling consistency and recognize domain-specific terms like 'Nunnari Academy'.



Transcription Parameters

Parameter	Type	Purpose	Example
model	string	Model to use	"whisper-1"
language	string	Hint the input language (ISO-639-1)	"ta" Tamil "hi" Hindi
prompt	string	Context to guide transcription	"Medical consultation recording"
response_format	string	Output format	"text" "srt" "verbose_json"
temperature	float	Sampling temperature	0 (most accurate)
timestamp_granularities	array	Word or segment timestamps	["word"] or ["segment"]

Why prompt matters

The model uses your prompt to maintain spelling consistency and recognize domain terms. Include names like 'Nunnari Academy' to avoid misspelling.

Temperature = 0

For transcription, lower temperature = more accurate. Set to 0 for deterministic output unless you need variation.



Output Formats

Format	Output Shape	Best For
text	Plain string	Simple display or database storage
json	{"text": "..."} with metadata	Programmatic access to result
verbose_json	Language, duration, word timestamps	Full analytics and search indexing
srt	SubRip subtitle format	Video captioning pipelines
vtt	WebVTT format	HTML5 video players

SRT Output (Subtitles)

```
transcript = client.audio
    .transcriptions.create(
        model='whisper-1',
        file=audio_file,
        response_format='srt'
    )
# → '1
# 00:00:00,000 --> 00:00:03,500
# Good morning everyone.
# 2 ...'
```

Verbose JSON (Timestamps)

```
for word in transcript.words:
    print(f'{word.word:15}'
          f'{word.start:.2f}s')
# → 'Good          0.00s'
# → 'morning       0.30s'
```



Translation & Large Audio Files

Any Language → English in One Call

```
# Tamil audio → English text
with open('tamil_meeting.mp3', 'rb') as f:
    translation = client.audio
        .translations.create(
            model='whisper-1',
            file=f,
            prompt='Business meeting about AI.'
        )
print(translation.text)
# → 'Today we will review the product
#    roadmap and discuss feedback...'
```

Chunking Strategy (>25MB)

```
from pydub import AudioSegment

def transcribe_long(path, mins=10):
    audio = AudioSegment.from_file(path)
    chunk_ms = mins * 60 * 1000
    text, prev = '', ''
    for i, start in enumerate(
        range(0, len(audio), chunk_ms)):
        chunk = audio[start:start+chunk_ms]
        chunk.export(f'c_{i}.mp3', format='mp3')
        with open(f'c_{i}.mp3', 'rb') as f:
            r = client.audio.transcriptions
                .create(model='whisper-1',
                    file=f, prompt=prev[-500:])
            text += r.text + ' '
            prev = r.text
    return text.strip()
```

Always pass the end of the previous chunk as the prompt for the next chunk — this prevents context loss at boundaries.

Whisper Limitations

Limitation	Workaround
25MB file size limit	Split audio into chunks using pydub
No speaker diarization	Use pyannote.audio or AssemblyAI for speaker ID
No real-time streaming	Use Deepgram or AssemblyAI for live transcription
Translation only to English	Chain with GPT for other target languages
Struggles with heavy accents	Provide language hint + prompt context

For most use cases, Whisper's limitations are manageable. For real-time streaming or diarization, pair Whisper with specialized tools.

The background is a solid teal color. It features a decorative pattern of small, dark teal dots scattered across the surface. A prominent line graph is overlaid on the background, consisting of dark teal circular nodes connected by straight lines. The graph starts at a low point on the left, rises to a peak, falls to a trough, and then rises again towards the right. The text is white and positioned on the left side of the image.

Part 2

TTS

(Text-to-Speech)



What is TTS?

OpenAI's TTS converts text into lifelike spoken audio using neural models. Natural intonation, pacing, and emotion — suitable for production applications.

Available Models

Model	Speed	Quality	Best For
tts-1	Fast, low latency	Standard	Real-time apps, chatbots, streaming
tts-1-hd	Slower	Higher	Podcasts, narration, repeated content

Use tts-1 when the user is waiting (latency matters). Use tts-1-hd for pre-generated content.

Available Voices — Choose Your Persona

Voice	Character	Best For
alloy	Neutral, balanced	General purpose, assistants
echo	Male, warm	Tutorials, educational content
fable	British, expressive	Storytelling, narration
onyx	Deep, authoritative	News, formal announcements
nova	Female, energetic	Customer service, marketing
shimmer	Female, soft	Meditation, wellness, calm content



Basic TTS & Parameters

```
from openai import OpenAI
from pathlib import Path

client = OpenAI()
response = client.audio.speech.create(
    model='tts-1',
    voice='nova',
    input='Welcome to Nunnari Academy.
    Lets begin todays session
    on Generative AI.'
)
response.stream_to_file('welcome.mp3')
```

Stream to File

`stream_to_file()` writes audio directly to disk — no intermediate buffers needed.

Voice Selection

Always test voices with your actual content. Best voice depends on use case and audience.

TTS Parameters — Full Reference

Parameter	Options	Purpose
model	tts-1, tts-1-hd	Quality vs speed trade-off
voice	alloy, echo, fable, onyx, nova, shimmer	Voice character and persona
input	string (max 4,096 chars)	Text to convert to speech
response_format	mp3, opus, aac, flac, wav, pcm	Output audio format
speed	0.25 to 4.0 (default 1.0)	Playback speed of audio



Output Formats & Streaming TTS

Format	Size	Best For
mp3	Medium	General use, web playback — default
opus	Smallest	Streaming, internet telephony
aac	Small	Mobile apps, iOS and Android
flac	Large	Lossless archival, audio production
wav	Largest	Uncompressed, audio processing
pcm	Raw bytes	Real-time streaming, custom playback

Streaming TTS — Real-Time

```
with client.audio.speech
    .with_streaming_response.create(
        model='tts-1',
        voice='nova',
        input='Here is your AI response.
              It begins playing immediately.',
        response_format='opus'
    ) as response:
        response.stream_to_file(
            'streamed_output.opus')
```

Streaming Latency

Configuration	First Audio Chunk
tts-1 + opus	~300–500ms
tts-1-hd + mp3	~800–1200ms

Use tts-1 + opus for lowest latency in conversational applications.



Speed Control & Limitations

Speed Control

```
# Slow down for language learning
response = client.audio.speech.create(
    model='tts-1', voice='echo',
    input='Tokenization breaks text into
          smaller units called tokens.',
    speed=0.75 # ← 25% slower
)

# Speed up for quick notifications
response = client.audio.speech.create(
    model='tts-1', voice='alloy',
    input='File processed successfully.',
    speed=1.25 # ← 25% faster
)
```

TTS Limitations

4,096 character limit

Split long text into chunks

No SSML support

Cannot control
pauses/emphasis

English-optimized

Other languages less natural

No emotion control

Cannot specify happy/sad
tone

No voice cloning

Limited to 6 built-in voices

~300ms min latency

Not instant for real-time

Speed range: 0.25x to 4.0x — great for educational content (slow) or quick notifications (fast).



Part 3

Building a Full Voice Pipeline



Whisper + GPT + TTS Voice Pipeline



```
def voice_assistant(audio_path, out='response.mp3'):
    # Step 1: Transcribe user voice -> text
    with open(audio_path, 'rb') as f:
        transcript = client.audio.transcriptions
            .create(model='whisper-1', file=f,
                    language='en')
        user_text = transcript.text

    # Step 2: Send to GPT for a response
    gpt_resp = client.chat.completions.create(
        model='gpt-4o-mini',
        messages=[
            {'role': 'system', 'content':
                'You are a helpful voice assistant.
                Be concise - responses spoken aloud.'},
            {'role': 'user', 'content': user_text}
        ], max_tokens=200)
    reply = gpt_resp.choices[0].message.content

    # Step 3: Convert response -> speech
    audio = client.audio.speech.create(
        model='tts-1', voice='nova', input=reply)
    audio.stream_to_file(out)
```

Keep It Short

Limit GPT responses to ~100 words.
Users cannot re-read audio. Use concise, conversational language.

System Prompt

Instruct GPT to be concise, avoid bullet points, and speak naturally — the output will be heard, not read.

Sub-2s Pipeline

Use Groq's Whisper + stream GPT tokens + begin TTS simultaneously for <1.5s total latency.



Latency Budget & Use Cases

Typical Latency Budget

Stage	Typical Latency
Whisper transcription	~1–3 seconds
GPT-4o-mini response	~0.5–2 seconds
TTS generation (tts-1)	~0.3–1 second
Total round trip	~2–6 seconds

Real-World Use Cases

Use Case	STT	LLM	TTS
Customer support voicebot	✓	✓	✓
Meeting transcription	✓	✓	✗
Language learning app	✓	✓	✓
Accessibility reader	✗	✓	✓
Podcast script to audio	✗	✗	✓
Multilingual support bot	✓	✓	✓
Voice-controlled agent	✓	✓	✓

For sub-2 second pipelines: use Groq's Whisper endpoint + stream GPT tokens + begin TTS streaming as tokens arrive.

Section Summary

The background is a solid purple color. It features a decorative pattern of small, dark purple dots arranged in a grid. Overlaid on this grid is a line graph consisting of larger, dark purple circular nodes connected by straight lines. The graph starts at a low point on the left, rises to a peak, falls to a trough, and then rises again towards the right. The text 'Section Summary' is written in a large, white, sans-serif font, positioned on the left side of the image and partially overlapping the graph's nodes.

Key Takeaways



Whisper (STT)

State-of-the-art speech recognition for 99 languages. Two modes — transcription (same language) and translation (to English). Use `verbose_json` for timestamps, `srt/vtt` for subtitles. \$0.006/min.



TTS

Neural text-to-speech with 6 voices and 2 models. Use `tts-1` for real-time, `tts-1-hd` for quality. Opus format for streaming. Speed control 0.25x–4.0x. 4,096 char limit.



Voice Pipeline

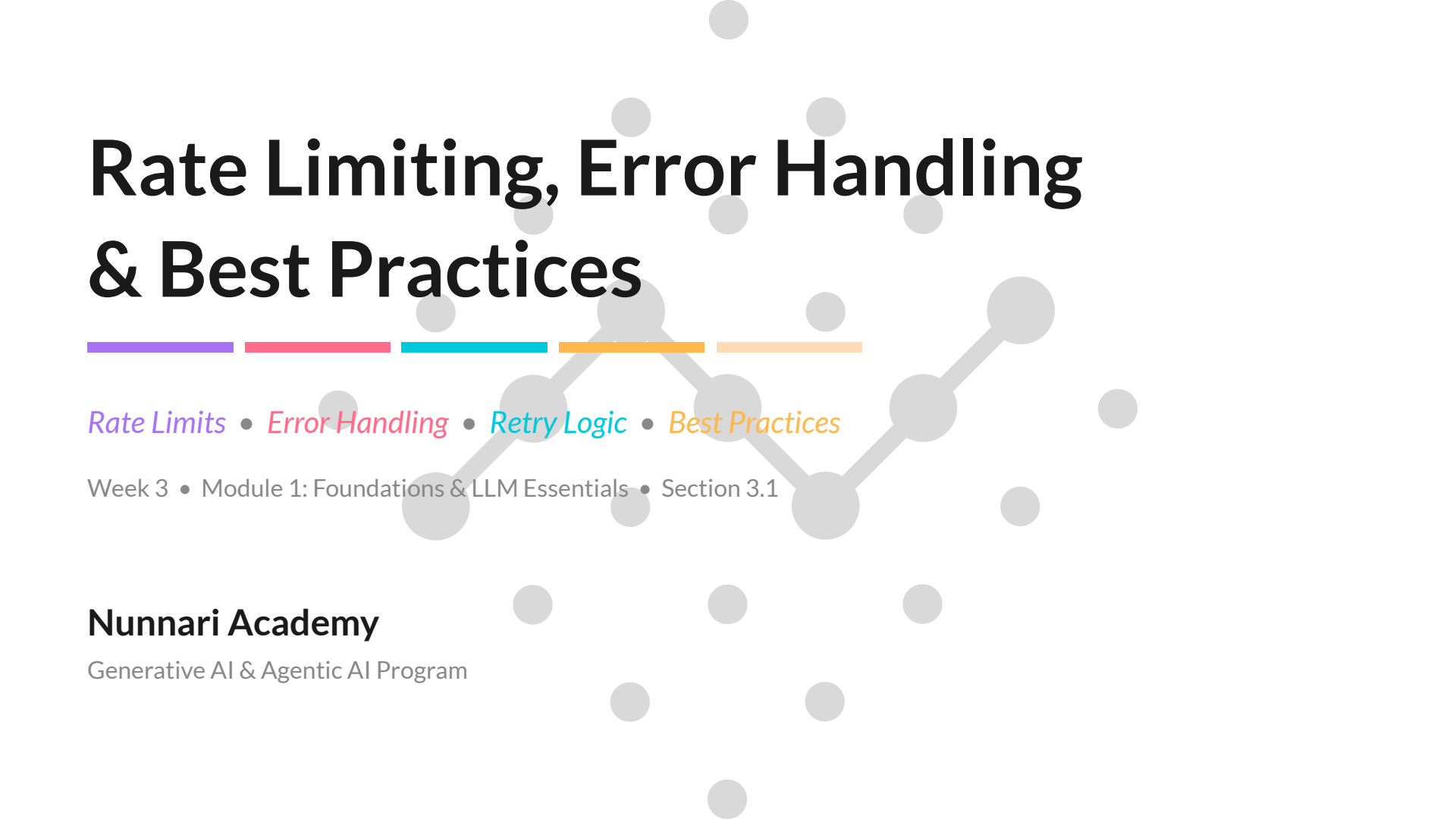
Whisper + GPT + TTS = complete voice assistant. Round-trip 2–6 seconds. Keep GPT responses short and conversational. Use Groq + streaming for sub-2s pipelines.



Cost Reference

Whisper: \$0.006/min. TTS: \$15/1M chars (`tts-1`) or \$30/1M chars (`tts-1-hd`). A 1-hour transcript costs ~\$0.36. A 1,000-word spoken response costs ~\$0.01.

Rate Limiting, Error Handling & Best Practices



Rate Limits • *Error Handling* • *Retry Logic* • *Best Practices*

Week 3 • Module 1: Foundations & LLM Essentials • Section 3.1

Nunnari Academy

Generative AI & Agentic AI Program



Overview — Four Core Areas

Building robust, production-ready LLM applications requires mastering rate limits, error handling, retry logic, and operational best practices.

Rate Limits

RPM, TPM, RPD
dimensions.
Tiered system that grows
with your usage.

Error Handling

Exception hierarchy.
Retriable vs non-retriable
error classification.

Retry Logic

Exponential backoff
with jitter. SDK built-in
vs tenacity library.

Best Practices

Tokens, timeouts, caching,
history management,
circuit breakers.

The background is a solid purple color. Overlaid on this are several decorative elements: a network of small, semi-transparent purple dots scattered across the frame; a line graph consisting of larger purple dots connected by thin, semi-transparent purple lines, showing a fluctuating trend; and the text 'Part 1' in a large, white, sans-serif font.

Part 1

Rate Limits



What are Rate Limits?

Rate limits restrict how many requests you can make within a time window. They ensure fair usage, prevent abuse, and maintain stability. Hitting a limit means your app needs to be smarter about when and how it sends requests.

Three Dimensions (Enforced Simultaneously)

Dimension	What It Measures	Example Limit
RPM	Requests Per Minute	500 requests/min
TPM	Tokens Per Minute	200,000 tokens/min
RPD	Requests Per Day	10,000 requests/day

A single large request might not exceed RPM but can exhaust TPM. Always monitor all three dimensions.

Rate Limit Headers — Real-Time Quota

```
x-ratelimit-limit-requests: 500 ← RPM ceiling
x-ratelimit-remaining-requests: 487 ← Requests left
x-ratelimit-remaining-tokens: 185432 ← Tokens left
x-ratelimit-reset-requests: 12s ← RPM resets in
x-ratelimit-reset-tokens: 0.5s ← TPM resets in
```



Rate Limit Tiers

Limits increase automatically as you spend more. No manual request needed — upgrades happen based on cumulative billing.

Tier	Requirement	GPT-4o-mini RPM	GPT-4o-mini TPM
Free	\$0 spent	3	40,000
Tier 1	\$5 paid	500	200,000
Tier 2	\$50 paid	5,000	2,000,000
Tier 3	\$100 paid	5,000	4,000,000
Tier 4	\$250 paid	10,000	10,000,000
Tier 5	\$1,000 paid	30,000	150,000,000

Tier 1 (\$5 paid) is sufficient for most learning projects and early-stage applications.



Part 2

Error Handling



The OpenAI Error Hierarchy

```
openai.OpenAIError      ← Base class
├── openai.APIError      ← Server-side
│   ├── openai.APIConnectionError ← Network/timeout
│   ├── openai.APITimeoutError ← Request timeout
│   └── openai.InternalServerError ← 500 error
├── openai.AuthenticationError ← 401 Invalid key
├── openai.PermissionDeniedError ← 403 No access
├── openai.NotFoundError ← 404 Wrong model
├── openai.RateLimitError ← 429 Rate limit
└── openai.BadRequestError ← 400 Bad params
```

Non-Retriable (4xx)

Auth, permission, not found, bad request errors. Never retry — fix the code or configuration.

Retriable (429, 5xx)

Rate limit, server errors, network failures. Always retry with exponential backoff + jitter.

Never lump all errors into a single catch-all block. Auth errors should never be retried — rate limit and server errors must always be.

Decision Framework

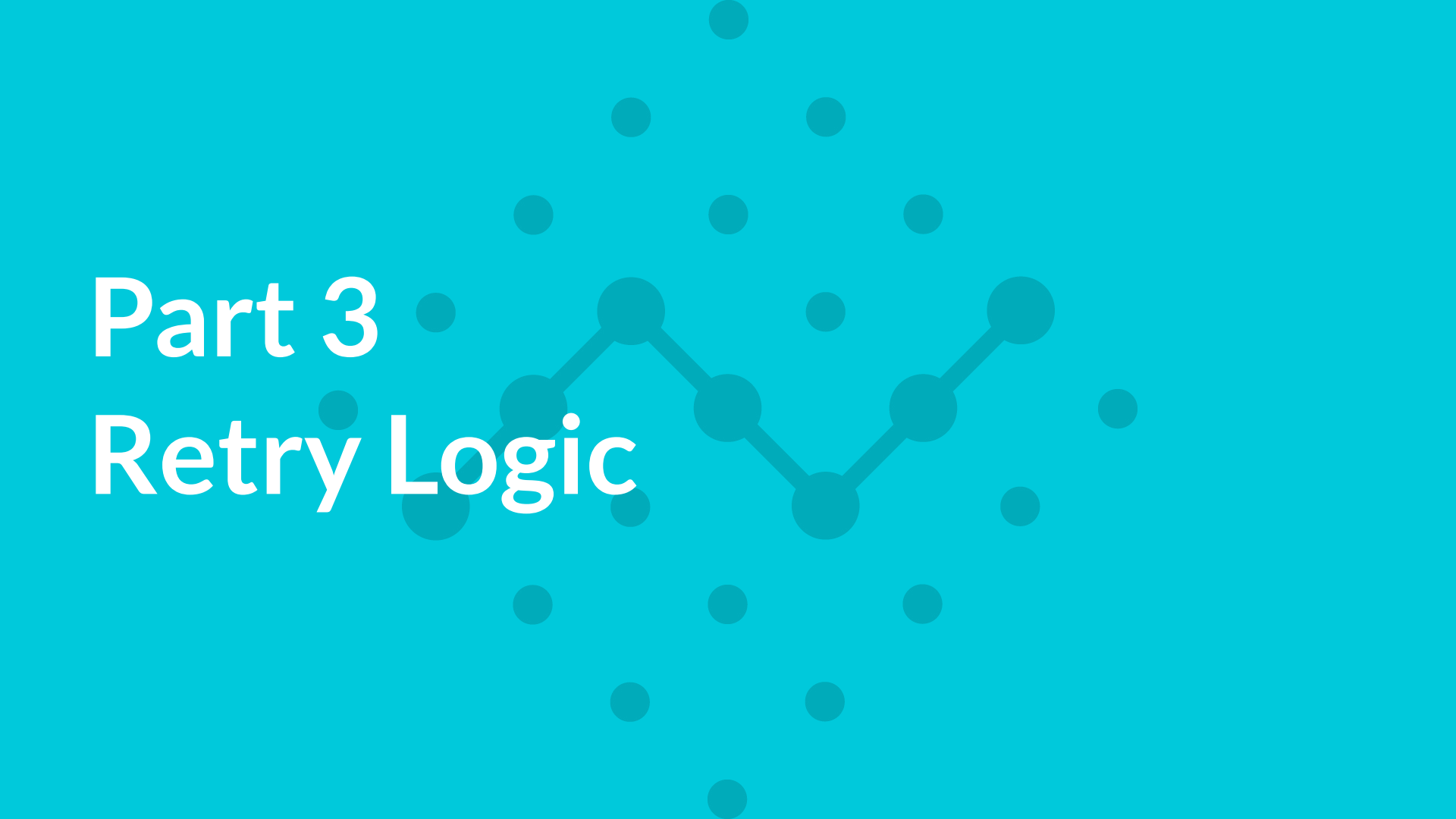
Error Category	Response Strategy
Non-retriable (4xx auth, bad req)	Log error, alert team, fail gracefully. Fix code.
Retriable (429, 5xx, network)	Backoff, retry up to max attempts, then raise.



Complete Error Reference

Error Class	HTTP	Cause	Retry?	Action
AuthenticationError	401	Invalid/missing API key	✗ Never	Check .env, regenerate
PermissionDeniedError	403	No access to model/org	✗ Never	Check org settings
NotFoundError	404	Wrong model name	✗ Never	Verify model string
BadRequestError	400	Invalid parameters	✗ Never	Fix request structure
RateLimitError	429	RPM/TPM/RPD exceeded	✓ Always	Backoff and retry
InternalServerError	500	OpenAI server issue	✓ Always	Retry after delay
APIConnectionError	—	Network failure	✓ Always	Check connectivity, retry
APITimeoutError	—	Request too slow	✓ Always	Retry, longer timeout

Top 4 rows: Never retry – fix your code. Bottom 4 rows: Always retry with exponential backoff.

The background is a solid teal color. It features a grid of small, darker teal dots. Overlaid on this grid is a line graph consisting of larger teal dots connected by lines, forming a jagged, upward-trending path.

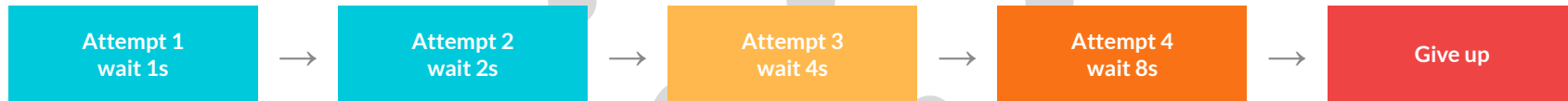
Part 3

Retry Logic



Exponential Backoff + Jitter

When a retriable error occurs, wait before retrying — each subsequent retry waits longer. This prevents hammering the API.



Always Add Jitter

Jitter is a random offset added to each wait. Without it, all clients retry at the same moment — causing another spike (thundering herd problem).

```
# wait = (2 ** attempt) + random(0,1)
# Attempt 1: 1.0s + 0.42s = 1.42s
# Attempt 2: 2.0s + 0.87s = 2.87s
# Attempt 3: 4.0s + 0.13s = 4.13s
```

Three Implementation Options

Option	Best For
SDK Built-in (max_retries=5)	Learning, prototyping
Manual Backoff Loop	Full per-error control
tenacity Library	Production systems

For learning: SDK max_retries. For production: tenacity with logging, alerting, and retry tracking.

Part 4

Best Practices

An abstract geometric pattern on an orange background. It features a central line graph with seven circular nodes connected by straight lines, forming a series of peaks and valleys. Surrounding this central structure are numerous smaller, isolated circular dots of varying sizes, scattered across the entire frame.



Best Practices 01–03

01 Always Set `max_tokens`

Without it, the model can generate up to its full context limit — potentially costing 10–100× more. Classification → 10 tokens. Summary → 300. Report → 2,000.

02 Monitor Token Usage on Every Response

Every response includes usage object with `prompt_tokens`, `completion_tokens`, `total_tokens`. Log this for every request — without it, you have no cost visibility.

03 Always Set Request Timeouts

Use Case	Recommended Timeout
Short queries — chat, classification	15–30 seconds
Long generation — reports, summaries	60–120 seconds
Streaming responses	120–300 seconds
Batch / background jobs	300+ seconds



Best Practices 04–05

04 Use Prompt Caching to Reduce Costs

OpenAI auto-caches prompt prefixes >1,024 tokens at 50% discount. Place static content (system prompt, instructions) at the START, dynamic content (user query) at the END.

What Breaks Prompt Caching

- ✗ Changing the system prompt between requests — even slightly
- ✗ Inserting dynamic values like timestamps into the static prefix
- ✗ Reordering messages between requests

A 2,000-token static prefix running 10,000 times/day saves ~\$30/day with caching vs uncached.

05 Count Tokens Before Sending

Use tiktoken to estimate token count before sending. Rule: leave 20% of context window for the response. Count tokens before sending user content, conversation history, and RAG document chunks.



Best Practices 06–07

06 Manage Conversation History

Strategy	How It Works	Best For
Sliding window	Keep only the last N messages	General chat apps
Token-based trim	Remove oldest until under limit	Precise cost control
Summarization	Compress old history to summary	Long sessions
RAG retrieval	Store externally, retrieve relevant	Domain assistants
<i>Always preserve the system prompt when trimming – only remove user and assistant messages.</i>		

07 Validate Outputs – Check `finish_reason`

<code>finish_reason</code>	Meaning	Action
stop	Completed naturally	✅ Use the response
length	Truncated by <code>max_tokens</code>	⚠️ Increase <code>max_tokens</code> or split
content_filter	Blocked by safety system	🚫 Review prompt, log event



Best Practices 08–09

08 Separate API Keys by Environment

Never use production keys in dev/testing. Use separate keys with separate spending limits.

- ✓ Runaway costs in dev do not affect production budget
- ✓ Revoke a dev key without disrupting production
- ✓ Usage logs stay clean — no test traffic pollution

09 Circuit Breakers for Production

Stops sending requests when too many fail — prevents cascading failures. Open after 5 consecutive failures. Wait 60s, then test one request. If it succeeds, resume.





Quick Reference Card

Practice	Priority	Why It Matters
Always set max_tokens	●	Prevents cost explosion
Set explicit timeouts	●	Prevents hung requests
Exponential backoff + jitter	●	Required for reliability
Check finish_reason	●	Catches truncated output
Log token usage per request	●	Cost visibility
Keep system prompts static	●	Enables prompt caching
Count tokens before sending	●	Prevents failures
Trim conversation history	●	Avoids context overflow
Separate keys per environment	●	Isolates risk
Implement circuit breakers	●	Production resilience
Never catch all exceptions	●	Correct error handling

Section Summary

The background is a solid purple color. It features a decorative pattern of small, dark purple dots arranged in a grid. Overlaid on this grid is a line graph consisting of larger, dark purple circular nodes connected by straight lines. The graph starts at a low point on the left, rises to a peak, falls to a trough, and then rises again towards the right. The text 'Section Summary' is written in a large, white, sans-serif font on the left side of the image.

Key Takeaways



Rate Limits

RPM, TPM, RPD enforced simultaneously. Read response headers for real-time quota. Limits auto-scale across 6 tiers based on cumulative spend.



Error Handling

Use specific exception hierarchy. Auth/bad request → never retry, fix code. Rate limit/server errors → always retry with backoff.



Retry Strategy

Exponential backoff with jitter prevents thundering herd. SDK max_retries for simple apps. tenacity for production.



Best Practices

Always set max_tokens and timeouts. Log token usage. Keep system prompts static for caching. Trim history. Separate keys. Check finish_reason.

Anthropic Claude API

Messages API & Unique Features

Claude Models • *Messages API* • *Unique Features* • *Claude vs GPT-4o*

Week 3 • Module 1: Foundations & LLM Essentials • Section 3.2

Nunnari Academy

Generative AI & Agentic AI Program



Overview — Four Core Areas

Claude is the top alternative to GPT-4o for enterprise workloads — especially where long context, nuanced reasoning, and instruction following matter most.

Anthropic & Claude

Safety-first AI company.
Constitutional AI training.
Three model tiers.

Messages API

Role-based interface.
Dedicated system param.
Strict role alternation.

Unique Features

200K context window.
Extended Thinking.
90% prompt caching.

Claude vs GPT-4o

Head-to-head comparison.
When to choose each.
Multi-provider strategy.



Part 1

Who is Anthropic?



Anthropic — Safety-First AI

Founded in 2021 by former OpenAI researchers, including Dario and Daniela Amodei. Their mission: build AI systems that are safe, beneficial, and understandable. Claude is their flagship AI assistant.

Anthropic vs OpenAI — At a Glance

	Anthropic	OpenAI
Founded	2021	2015
Core focus	AI Safety research	AI capability + products
Philosophy	Constitutional AI	RLHF-driven
Model family	Claude (Opus/Sonnet/Haiku)	GPT (o-series/4o/mini)
Flagship	Claude Opus 4	GPT-4o / o1
Key backing	Amazon, Google	Microsoft

In production, teams rarely commit to a single provider. Claude is the top alternative to GPT-4o for enterprise workloads.



Claude Model Family

Three-tier naming convention mapping directly to the intelligence-vs-cost trade-off:

Model	Tier	Best For	Context
Claude Opus	Most capable	Complex reasoning, research, nuanced analysis	200K
Claude Sonnet	Balanced	Production workloads, everyday tasks	200K
Claude Haiku	Fastest & cheapest	Classification, quick responses, high volume	200K

All Claude models share the same 200K token context window – one of the largest in the industry (vs GPT-4o's 128K).

Current Model Versions (2025)

Claude Opus 4	Deepest reasoning, research and complex multi-step tasks
Claude Sonnet 4	Best performance-to-cost ratio for production apps
Claude Haiku 3.5	Fastest response, real-time and high-volume apps

The background is a solid teal color. It features a pattern of smaller, darker teal dots scattered across the surface. Overlaid on this is a faint, stylized network diagram consisting of several circular nodes connected by thin, light-colored lines, forming a web-like structure.

Part 2

The Messages API



Messages API – Request Structure

```
import anthropic

client = anthropic.Anthropic(
    api_key='your-api-key')

message = client.messages.create(
    model='claude-sonnet-4-5',
    max_tokens=1024,
    system='You are a helpful AI tutor
    for Nunnari Academy.',
    messages=[
        {'role': 'user',
         'content': 'Explain transformers
         in simple terms.'}
    ]
)
print(message.content[0].text)
```

system = Top-Level Param

System prompt is a dedicated parameter — not inside the messages array. Anthropic treats it as fundamentally different from conversation.

Strict Role Alternation

Messages must alternate user → assistant → user. Two consecutive messages from the same role will cause an error.

Response Access

message.content[0].text (not .choices[0].message.content like OpenAI).

Key Differences from OpenAI

Feature	Claude	GPT
System prompt	Separate system param	Message with role: 'system'
Response	message.content[0].text	.choices[0].message.content
Stop reason	message.stop_reason	.choices[0].finish_reason



Response Object & stop_reason

Response Object — Key Fields

Field	What It Contains
message.id	Unique message identifier for logging
message.model	The model that generated the response
message.content[0].text	The actual response text
message.stop_reason	"end_turn" "max_tokens" "stop_sequence"
message.usage.input_tokens	Tokens consumed by your prompt
message.usage.output_tokens	Tokens generated in the response

stop_reason Values

Value	Meaning	Action
end_turn	Model finished naturally	✅ Use the response
max_tokens	Hit the token limit	⚠️ Increase max_tokens

✅ Expected behaviour

Multi-turn: pass full history each request

```
messages = [  
  {'role': 'user',      'content': 'What is RAG?'},  
  {'role': 'assistant', 'content': 'RAG stands for...'},  
  {'role': 'user',      'content': 'How does it differ?'}  
]
```



Part 3

Claude's Unique Features



Constitutional AI – Safety by Design

01

Claude is trained against a written 'constitution' – a set of principles about being helpful, harmless, and honest – rather than relying solely on human feedback like OpenAI's RLHF.

Aspect	OpenAI (RLHF)	Anthropic (CAI)
Safety training	Human labeler feedback	Written constitution + AI self-critique
Refusals	Can feel inconsistent	Principled, predictable
Ethics reasoning	Implicit in training	Explicit constitutional rules
Transparency	Limited visibility	Published principles

Constitutional AI makes Claude's refusals predictable and explainable – critical for compliance-sensitive industries like healthcare, finance, and legal.

02

200K Token Context Window

Tokens	Approximate Content
1,000	~750 words (one page)
50,000	~100 pages (a novel)
200,000	~300+ pages (multiple books / full codebase)



200K Context — What It Unlocks

✓ Analyzing entire legal contracts in one pass — no chunking needed

✓ Loading an entire codebase for debugging and refactoring

✓ Processing full research papers with all citations intact

✓ Summarizing multi-day meeting transcripts end to end

✓ Running RAG with much larger retrieved contexts per query

Longer context = fewer chunking errors, better cross-document reasoning, simpler architecture. For many use cases, 200K eliminates the need for RAG entirely.



Tool Use & Vision Capabilities

03 Tool Use (Function Calling)

Concept	OpenAI	Claude
Definition field	parameters (JSON Schema)	input_schema (JSON Schema)
Tool call in response	tool_calls array	tool_use content block
Tool result role	role: 'tool'	role: 'user' with tool_result
Multiple tools	Parallel calls supported	Parallel calls supported

04 Vision Capabilities

Feature	Claude	GPT-4V
Image input	content array (base64/URL)	content array (base64/URL)
URL images	✅ Supported	✅ Supported
Base64 images	✅ Supported	✅ Supported
Detail control	Not available	low/high/auto
Max images/req	20 images	Multiple (varies)



Extended Thinking — Reasoning Mode

05

Claude Opus 4 supports extended thinking — the model reasons step by step before generating its final answer (similar to OpenAI's o1). The thinking field sets a token budget for internal reasoning.

When to Use Extended Thinking

Use Case	Standard Claude	Extended Thinking
Simple Q&A	✓ Sufficient	Overkill
Complex math	May struggle	✓ Recommended
Multi-step planning	Basic capability	✓ Recommended
Code debugging	Good for simple bugs	✓ Complex debugging
Strategic analysis	Surface-level	✓ Deep analysis

Higher thinking budgets allow deeper analysis but increase cost and latency. Use only when task complexity justifies it.



Prompt Caching — Explicit Control

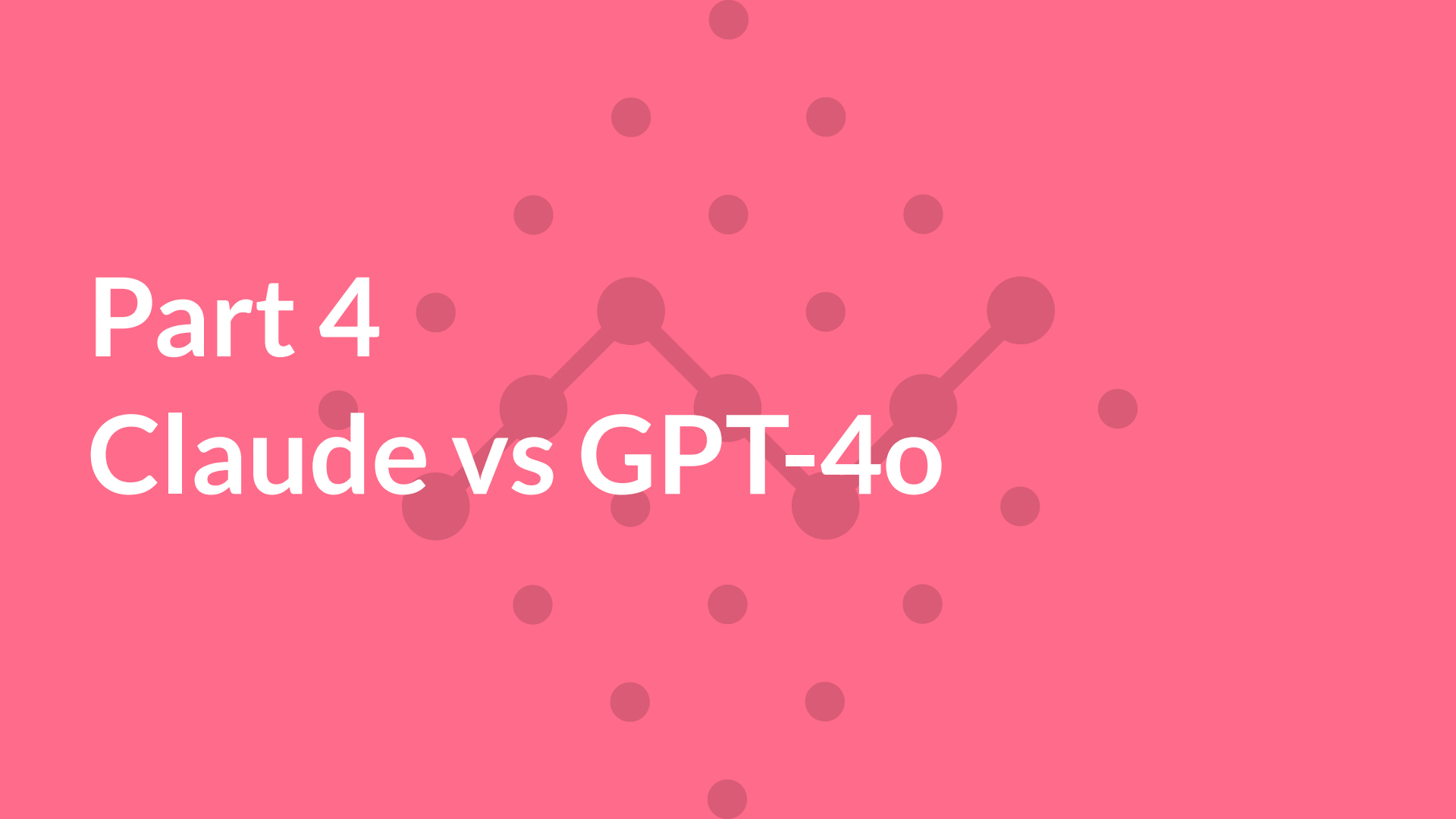
06

Claude supports prompt caching with explicit control — you mark which parts should be cached using a `cache_control` field, giving precise control over what gets stored and reused.

Claude Caching vs OpenAI Caching

Feature	Claude	OpenAI
Cache control	Explicit (<code>cache_control</code> field)	Automatic (prefix matching)
Min cacheable tokens	1,024 tokens	1,024 tokens
Cache discount	90% off cached input	50% off cached input
Cache lifetime	5 minutes	5–60 minutes
Cost to write cache	25% surcharge on first write	No surcharge

Claude's 90% cache discount is far more aggressive than OpenAI's 50%. For apps repeatedly processing large documents (legal, medical, financial) — costs drop 80–90% per query after the first request.



Part 4

Claude vs GPT-4o



Claude vs GPT-4o — Head-to-Head

Dimension	Claude Sonnet 4	GPT-4o
Context window	200K tokens	128K tokens
Instruction following	Excellent — strict adherence	Very good
Code generation	Strong	Strong
Reasoning	Deep (Extended Thinking)	Strong (o1 for complex)
Safety / refusals	Principled (Constitutional AI)	RLHF-based
Multimodal	Text + Image	Text + Image + Audio
Audio support	✗ Not available	✓ Native (Whisper + TTS)
Extended thinking	✓ Opus 4	✓ o1 model
Prompt cache discount	90%	50%
Pricing (input)	\$3 / 1M tokens	\$2.50 / 1M tokens
Pricing (output)	\$15 / 1M tokens	\$10 / 1M tokens



When to Choose Which?

Choose Claude When

- Very long documents — contracts, books, codebases
- Extended thinking for complex reasoning
- Compliance-sensitive domains (healthcare, legal)
- Maximum caching discount (90% vs 50%)
- Strict instruction following is critical

Choose GPT-4o When

- Audio input/output needed natively
- Team deeply familiar with OpenAI ecosystem
- Widest range of tools, plugins, integrations
- Output cost is primary concern at scale

Use Both When

- Multi-provider systems for resilience and cost optimization
- A/B testing model quality for your specific domain
- Failover capability — route to the other on outage

The most robust production architectures treat LLM providers like cloud providers — abstract behind a thin provider layer so you can switch or balance without rewriting logic.

Section Summary

The background is a solid purple color. It features a decorative pattern of small, dark purple dots arranged in a grid-like fashion. Overlaid on this pattern is a line graph consisting of several dark purple circular nodes connected by straight lines. The graph starts at a low point on the left, rises to a peak, falls to a trough, and then rises again towards the right. The text 'Section Summary' is written in a large, white, sans-serif font on the left side of the image.

Key Takeaways



Anthropic & Claude

Safety-first AI company using Constitutional AI. Three tiers — Opus (most capable), Sonnet (balanced), Haiku (fastest). All share a 200K token context window.



Messages API

System prompt is a dedicated top-level parameter. Response via `message.content[0].text`. Stop reason via `message.stop_reason`. Strict user/assistant alternation.



Unique Features

200K context (vs 128K), Constitutional AI safety, explicit prompt caching with 90% discount (vs 50%), Extended Thinking for deep reasoning, tool use with `input_schema`.



Claude vs GPT-4o

Claude wins on context length, instruction following, and caching economics. GPT-4o wins on audio, ecosystem breadth, and output pricing. In production — use both.

Google Gemini API



Multimodal Capabilities • Native AI Stack

Week 3 • Module 1: Foundations & LLM Essentials • Section 3.2

Nunnari Academy

Generative AI & Agentic AI Program

Key Topics



Google DeepMind

Full-stack AI advantage,
custom TPUs, and the
Gemini model family



Native Multimodal

Text, image, audio, video,
PDF, and code — all in
one unified API call



API & Capabilities

SDK structure, multi-turn
chat, search grounding,
and code execution

Google DeepMind & Gemini

The background is a solid purple color. It features a decorative pattern of small, semi-transparent purple dots scattered across the surface. A prominent graphic element is a line graph or path composed of larger, solid purple circles connected by thin, solid purple lines. This path starts from the left side, moves upwards and to the right, then downwards and to the right, and finally upwards and to the right again, ending towards the right edge of the frame.



Who is Google DeepMind?

Gemini is Google's flagship multimodal AI family — formed in 2023 by merging Google Brain and DeepMind. Google owns the entire AI stack: hardware to deployment.

Google's AI Advantage

Advantage	What It Means
Custom TPUs	Trains and serves models on Google's own Tensor Processing Units
Search & Knowledge	Access to the world's largest indexed knowledge base during training
Google Workspace	Native integration with Gmail, Docs, Sheets, and Drive
Cloud Infrastructure	Vertex AI provides enterprise-grade deployment out of the box
Multimodal by Design	Built multimodal from day one — not retrofitted after the fact

Key Distinction

GPT-4 and Claude were text models that gained vision.

Gemini was designed from the ground up to natively understand and generate across text, images, audio, video, and code simultaneously — a fundamentally different architecture.

Current Flagship Models (2025)

Gemini 2.0 Pro — Best for production workloads

Gemini 2.0 Flash — Best speed-to-quality ratio

Gemini 2.0 Flash Thinking — Reasoning with chain-of-thought



Gemini Model Family

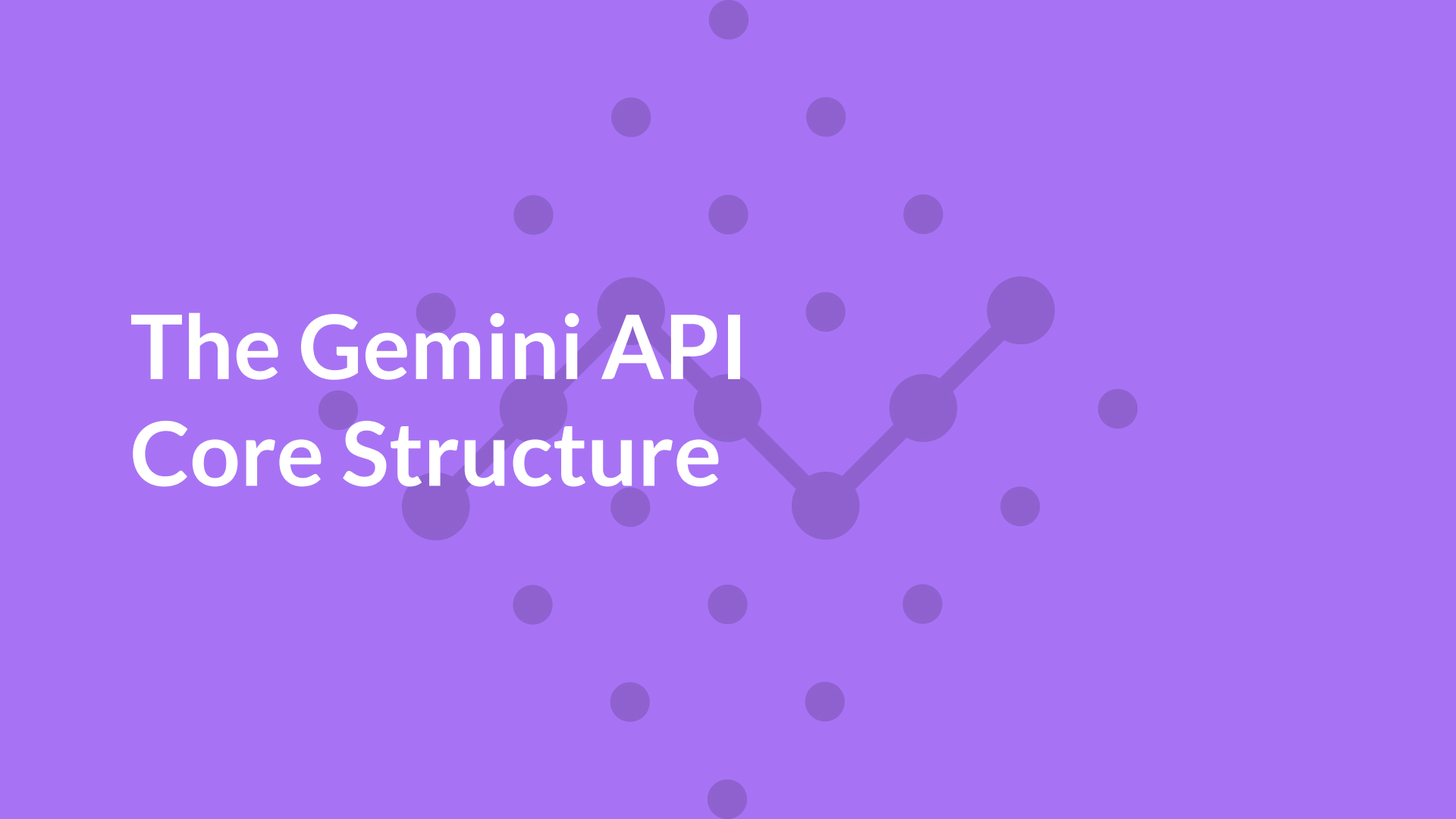
Gemini follows the same three-tier naming pattern as Claude – Ultra/Pro/Flash mapping directly to capability vs cost.

Model	Tier	Context Window	Best For
Gemini Ultra	Most capable	1M tokens	Complex reasoning, research, multimodal analysis
Gemini Pro	Balanced	1M tokens	Production workloads, everyday tasks
Gemini Flash	Fastest & cheapest	1M tokens	Real-time apps, high-volume, simple tasks
Gemini Flash 8B	Ultra-lightweight	1M tokens	Edge deployments, extremely high volume



1 Million Token Context Window At 1M tokens, you can load approximately 750,000 words – equivalent to 7-8 full novels, an entire software repository, or hours of video transcripts – into a single context. This is 5x Claude's 200K and 8x GPT-4o's 128K.

The Gemini API Core Structure

The background is a solid purple color. It features a decorative pattern of small, semi-transparent purple dots scattered across the surface. A prominent line graph, also in a semi-transparent purple, is overlaid on the right side of the image. The graph consists of several circular nodes connected by straight lines, showing an upward trend from left to right.



The Gemini API – Core Structure

Google provides two access methods: Google AI SDK for developers and Vertex AI for enterprise production.

Google AI SDK vs Vertex AI

Feature	Google AI SDK	Vertex AI
Target	Developers, prototyping	Enterprise production
Auth	API key	Service account / OAuth
Privacy	Standard terms	Enterprise data handling
SLA	Best-effort	99.9% uptime

Basic Request Structure

```
import google.generativeai as genai

genai.configure(api_key='your-key')

model = genai.GenerativeModel(
    'gemini-2.0-flash'
)

response = model.generate_content(
```

 **Key API Difference** Gemini uses `GenerativeModel + generate_content()` vs OpenAI's `client.chat.completions.create()` and Claude's `client.messages.create()`. Response accessed via `response.text` (not `response.choices[0].message.content`).

Native Multimodal Capabilities

The background is a solid purple color. It features a network of dark purple dots of varying sizes. Some dots are connected by thin, dark purple lines, forming a web-like structure. The dots are scattered across the entire frame, with a higher concentration of connected dots in the center-right area.



What Makes Gemini Multimodal?

Gemini natively encodes and reasons across modalities in a unified representation space — not a feature addition, but a core design choice.

Modality	What Gemini Can Do
 Text	Generation, summarization, translation, reasoning
 Images	Understanding, description, OCR, diagram analysis
 Audio	Transcription, speaker detection, tone analysis
 Video	Frame-level understanding, temporal reasoning, summarization
 Code	Generation, debugging, explanation, sandboxed execution
 Documents	PDF reading, table extraction, structured understanding



Unified Processing

Gemini processes all modalities simultaneously. You can send a video, ask about a specific moment, reference a chart in the video, and get a cross-temporal answer — in a single API call.

Two Ways to Send Images

Inline Data

Base64 encoded, small files

File API

Upload once, reuse URI 48hrs

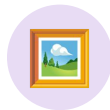


Image & Audio Understanding

Image Capabilities

Object & scene recognition — Identifies objects, people, settings

Text extraction (OCR) — Reads printed and handwritten text

Chart & graph analysis — Extracts data trends and values

Document understanding — Reads forms, invoices, docs

Spatial reasoning — Understands layout and position

🎵 **Gemini vs Whisper** Gemini handles audio in the same `generate_content()` call — no separate Whisper-like service needed. Plus native speaker diarization and follow-up reasoning.

Audio Capabilities

Speech transcription — Converts spoken audio to text

Speaker diarization — Separates different speakers

Tone & sentiment — Detects emotional tone and urgency

Language detection — Identifies spoken language auto

Multi-language audio — Handles code-switching

💡 **Gemini's Audio Edge** You can ask questions ABOUT the audio, not just transcribe it. It reasons over the audio content.



Video Understanding – Gemini's Signature

Video understanding is where Gemini truly differentiates itself. While GPT-4o and Claude handle static images, Gemini processes and reasons about video across time.

Video Summarization

"5-point summary of a 30-min lecture"

Temporal Reasoning

"When does the speaker contradict?"

Object Tracking

"How does the product change?"

Action Recognition

"List every action in this tutorial"

Scene Segmentation

"Break video into chapters + timestamps"

Multi-Video Compare

"Compare presentation styles in two pitches"



How Gemini Processes Video Rather than analyzing every frame, Gemini samples at ~1 fps and processes the audio track separately. For precise frame-level analysis, specify timestamps directly in your query. No other major provider offers native video understanding.



Document Understanding

Gemini natively reads PDFs without extracting text first – understanding layout, structure, embedded images, tables, and visual hierarchy all at once.

Gemini Native PDF vs Text Extraction

Feature	Gemini Native	Text Extraction
Embedded images	✓	✗
Table understanding	✓	⚠ Limited
Visual layout	✓	✗
Scanned PDFs (OCR)	✓	✗
Cross-page reasoning	✓	✗

Practical Use Cases

Contract Analysis Read clauses, identify risk areas

Financial Reports Extract tables, charts, narrative

Academic Papers Summarize methodology & results

Form Processing Extract data from unstructured forms

Invoice Extraction Handle scanned or digital invoices

Multi-Turn Chat & Additional Capabilities



Multi-Turn Conversations – Stateful Chat

Gemini's SDK manages conversation history internally through a chat session object – the most significant API difference from OpenAI and Claude.

OpenAI / Claude – Manual History

```
messages = [  
    {'role': 'user',  
     'content': '...'},  
    {'role': 'assistant',  
     'content': '...'},  
    {'role': 'user',  
     'content': '...'}  
    # Must include full history  
    # every time  
]
```

Gemini – Automatic History

```
chat = model.start_chat()  
  
chat.send_message('Hello')  
# History stored internally  
  
chat.send_message('Follow up')  
# Previous context auto-included  
  
print(chat.history)  
# Access full history anytime
```

History

Gemini: SDK manages automatically

OpenAI/Claude: You manage manually

Simplicity

Gemini: Very easy – just `send_message()`

OpenAI/Claude: Must rebuild messages array

Flexibility Conversations

Gemini: Less – harder to edit history

OpenAI/Claude: Full control over context



Additional Capabilities

01

Grounding with Google Search

Gemini can ground responses with live Google Search results — reducing hallucinations. Enable in model config, no extra API call needed. Responses include citations automatically.

02

Code Execution

Write and execute Python in a sandboxed environment. Solves math by running actual calculations. Generates and runs data analysis code. Validates its own solutions before responding.

03

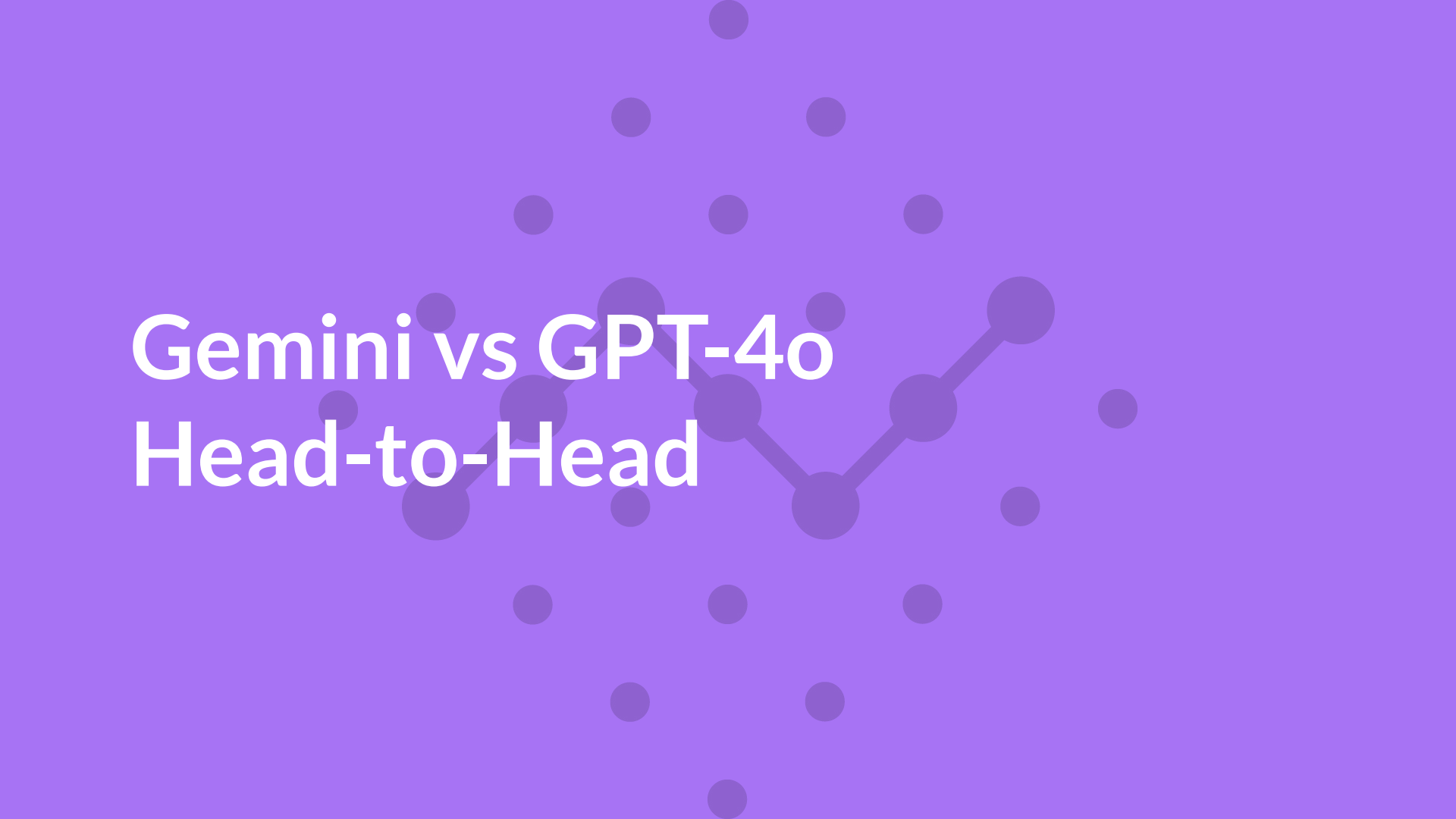
Structured Output & JSON Mode

Set `response_mime_type` to 'application/json' and provide a `response_schema`. Gemini guarantees the response matches the schema. Same pattern as OpenAI and Claude.

04

Embeddings

`text-embedding-004` produces 768-dimension vectors through the same SDK. Use for semantic search, document clustering, and RAG pipelines.

The background is a solid purple color. It features a decorative pattern of small, semi-transparent purple dots scattered across the surface. A prominent line graph is overlaid on the background, consisting of a series of connected dots of varying sizes and a line that trends upwards from left to right.

Gemini vs GPT-4o

Head-to-Head

VS

Gemini vs GPT-4o – Head-to-Head

Dimension	Gemini 2.0 Pro	GPT-4o
Context window	1M tokens ✓	128K tokens
Image understanding	Native multimodal	Vision API
Audio (native)	✓ Built-in	Whisper (separate)
Video (native)	✓ Signature feature	✗ Not supported
PDF (native)	✓ Full understanding	Text extraction only
Code execution	✓ Sandboxed Python	✓ Code Interpreter
Search grounding	✓ Google Search built-in	Via plugins/browsing
Conversation state	SDK-managed (stateful)	Manual (stateless)
Ecosystem	Google Workspace	✓ Broadest plugins



Gemini wins on context (1M), video, native PDF, pricing & Google integration. GPT-4o wins on ecosystem breadth, instruction following & enterprise maturity.



When to Choose Gemini

Choose Gemini When

- ✓ Your use case involves video content
- ✓ You need the longest context window (1M tokens)
- ✓ Building on Google Cloud / Vertex AI
- ✓ You need native PDF understanding
- ✓ You want Google Search grounding
- ✓ Building Google Workspace integrations

Choose GPT-4o When

- 📦 Deeply invested in OpenAI ecosystem
- 📦 Need widest range of 3rd-party integrations
- 📦 Strong instruction following is critical
- 📦 Need most mature enterprise compliance

💡 **Production Recommendation** Use Gemini for video, long documents, or Google Workspace pipelines. Use GPT-4o for broadest ecosystem support. The best architectures are provider-agnostic — abstract your LLM calls to route based on task type and cost.

Section Summary

The background is a solid purple color. It features a decorative pattern of small, light purple dots arranged in a grid. Overlaid on this is a white line graph with several circular nodes connected by straight lines, forming a jagged path across the center of the image.

Section Summary



Google DeepMind & Gemini: Multimodal-first AI from the team that owns the entire stack. Three tiers: Ultra, Pro, Flash. All share a 1M token context window — 8x GPT-4o's 128K.



Native Multimodality: Text, image, audio, video, PDF, and code in a single `generate_content()` call. No separate endpoints. Built multimodal from day one.



Video — Gemini's Signature: Temporal reasoning, scene segmentation, action recognition across video. No other major provider offers this natively.

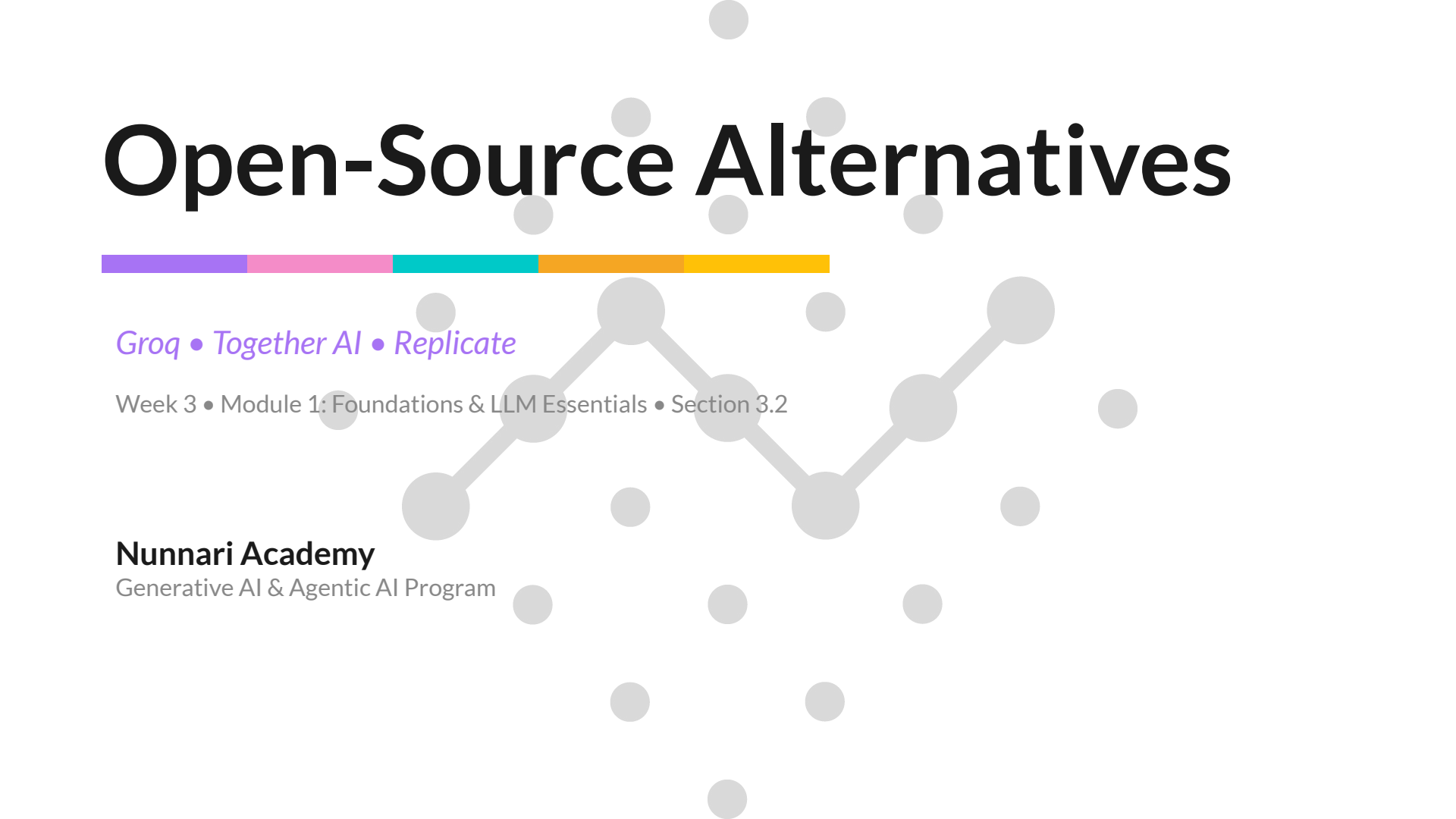


API & Capabilities: Stateful chat sessions, Google Search grounding, sandboxed code execution, and JSON mode — all in one SDK.



Gemini vs GPT-4o: Gemini wins on context (1M), video, native PDF, pricing. GPT-4o wins on ecosystem breadth and enterprise maturity. Use both based on task.

Open-Source Alternatives



Groq • Together AI • Replicate

Week 3 • Module 1: Foundations & LLM Essentials • Section 3.2

Nunnari Academy

Generative AI & Agentic AI Program

The Three Providers



Speed-First Inference

Custom LPU hardware,
800–1,200 tok/sec,
fastest public inference



Full-Stack Platform

100+ models, fine-tuning,
image gen, LLaMA 405B,
complete ecosystem



Deploy Any Model Zero DevOps

10,000+ models, all
modalities, per-second
billing, Cog deployment



Why Open-Source LLM Providers?

The major commercial providers come with trade-offs: data passes through their systems, pricing scales, and you have no control over the model. Open-source offers a different path.

Benefit	What It Means
Model transparency	You know exactly what model is running — weights are publicly available
Data control	Sensitive data stays within your infrastructure choices
Cost at scale	Significantly cheaper per token at high volumes
Model variety	Access to hundreds of community fine-tuned models
No vendor lock-in	Switch models or providers without rewriting your application
Offline capability	Run models locally with Ollama — zero API dependency



The Ecosystem Has Matured

Models like LLaMA 3.1 405B, Mixtral 8x22B, and Qwen 2.5 now compete directly with GPT-4o on many benchmarks — at a fraction of the cost.

Open-source is no longer a compromise — it is a serious production choice.

The Three Roles

Groq Fastest token generation

Together AI Widest model selection + training

Replicate Any model, zero DevOps



Groq Speed-First Inference

Groq — Custom LPU Hardware

Groq is not a model company — it is an inference hardware company. Their Language Processing Unit (LPU) was built from scratch for autoregressive token generation.

Inference Speed Comparison

Provider	Tokens/Second	Latency to First Token
 Groq (LLaMA 70B)	800–1,200 tok/sec	~100–200ms
Together AI	100–200 tok/sec	~200–400ms
OpenAI GPT-4o	80–120 tok/sec	~400–800ms
Standard GPU cloud	50–100 tok/sec	~500ms–1s

 **The Result** 10–20x faster than GPU-based inference. At 1,200 tok/sec, a 500-word response arrives in under 500 milliseconds.

When to Use Groq

-  Real-time voice assistants
-  Live coding assistants & IDE integrations
-  Agentic pipelines with many sequential calls
-  Streaming chat interfaces
-  High-throughput batch inference

Available Models

LLaMA 3.3 70B • LLaMA 3.1 8B • Mixtral 8x7B
Gemma 2 9B • Whisper Large v3



Together AI Full-Stack Platform



Together AI – The Full-Stack Platform

The most comprehensive open-source LLM platform: fast inference, fine-tuning, custom model deployment, and the widest model selection available.

Model Catalog

Category	Notable Models
General purpose	LLaMA 3.3 70B, LLaMA 3.1 405B, Qwen 2.5 72B
Coding	DeepSeek Coder V2, Qwen 2.5 Coder 32B
Reasoning	DeepSeek R1, QwQ 32B
Vision	LLaMA 3.2 Vision 90B, Qwen VL
Embeddings	BGE-M3, UAE-Large, BAAI embeddings
Image gen	FLUX.1, Stable Diffusion XL, SDXL Turbo

 **LLaMA 3.1 405B Available** The largest open-source model – rivaling GPT-4o – at open-source pricing. Not available on Groq, limited elsewhere.

When to Use Together AI

-  Fine-tuning on custom proprietary data
-  Image generation alongside text models
-  Vision-language models (open-source)
-  Access to LLaMA 3.1 405B
-  Embeddings + LLMs from one provider
-  Experimenting across model architectures



Together AI – Fine-Tuning Workflow

Together AI's signature differentiator: fine-tune any supported open-source model on your own data directly through their platform.

Fine-Tuning Steps

1

Prepare dataset in JSONL format Instruction + response pairs

2

Upload dataset via dashboard or API To Together AI platform

3

Launch fine-tuning job Choose base model + hyperparameters

4

Monitor training progress Via dashboard in real-time

5

Deploy as private endpoint Your own hosted model

6

Query via standard API Same interface as any model

Fine-Tuning Use Cases

Customer Support

Train on your company's historical support tickets

Code Assistant

Fine-tune on your internal codebase conventions

Domain Expert

Medical, legal, or financial specialized model

Brand Voice

Write in your brand's specific voice and tone



Replicate
Deploy Any Model



Replicate — The App Store for AI Models

Think of Replicate as the App Store for AI models — anyone can publish, you can run any published model with the same API call structure. Zero infrastructure management.

What Makes Replicate Unique

Feature	Detail
Universal model hub	10,000+ community and official models
Any modality	Text, image, audio, video, 3D, code — all in one
Zero DevOps	No GPU management, scaling, or config
Pay-per-second	Billed for actual compute time — not per token
Custom deploy	Deploy own models via Cog packaging tool
Versioned models	Every version is immutable — fully reproducible

 **Replicate's Unique Edge** Generate an image, upscale it, apply ControlNet, generate background music, and produce a video — all through the same API with the same call structure.

Beyond Text — All Modalities

Text LLMs LLaMA 3.1, Mistral, Mixtral

Image Gen FLUX.1, SD3, SDXL, ControlNet

Image Editing InstructPix2Pix, Real-ESRGAN

Audio Whisper, MusicGen, AudioCraft

Video Gen Stable Video Diffusion, CogVideoX

3D Gen TripoSR, Point-E



Replicate — Deployment & Pricing

Deploy Your Model with Cog

1

Install Cog CLI — On your local machine

2

Write predict.py — Define inputs and outputs

3

cog build — Creates reproducible Docker container

4

cog push — Deploys to Replicate infrastructure

5

Query via API — Standard Replicate API endpoint

Pricing — Pay Per Second

Hardware	Typical Use
Nvidia T4	Small models, light tasks
Nvidia A40	Mid-range inference
Nvidia A100	Large models, image gen
8× A100	Largest models (70B+)

When to Use Replicate



Image, audio, video, or 3D generation



Rapid prototyping without infra setup



Custom model hosting without DevOps

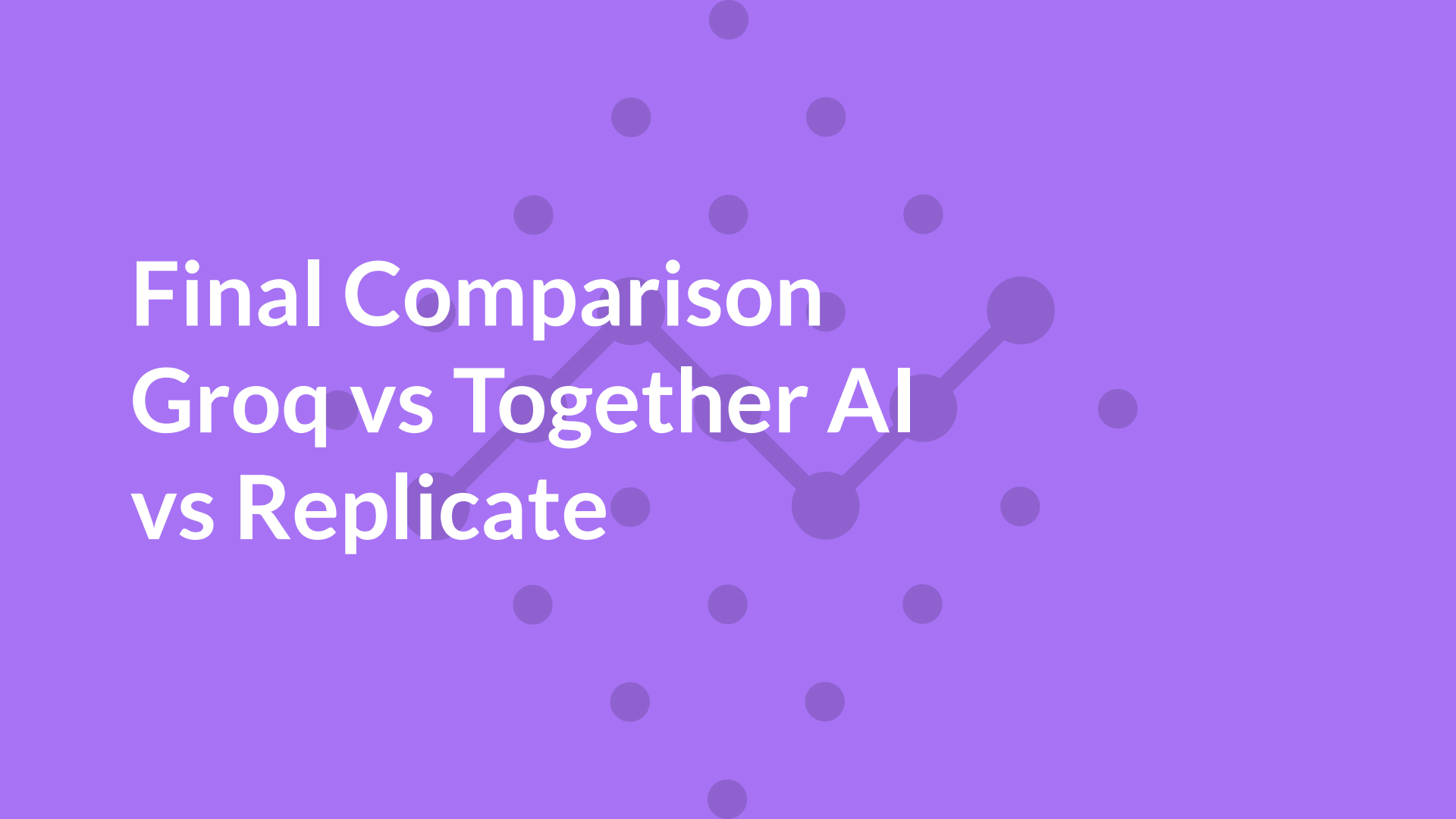


Multi-modal pipelines in same API



Community fine-tuned model variants

Nunnari Academy



Final Comparison Groq vs Together AI vs Replicate



Head-to-Head Comparison

Dimension	⚡ Groq	👉 Together AI	🚀 Replicate
Primary strength	Raw speed	Full-stack platform	Model variety
Core technology	Custom LPU chips	GPU clusters	GPU cloud + Cog
Inference speed	⚡ Fastest (10-20x)	Fast	Variable
Model count	~15 curated	100+	10,000+
LLaMA 405B	✗	✓	✓ (community)
Image generation	✗	✓ FLUX, SDXL	✓ Widest selection
Video / Audio / 3D	Whisper only	✗	✓ All modalities
Fine-tuning	✗	✓ Full platform	✗ (deploy only)
Custom deploy	✗	✓	✓ Cog
OpenAI-compatible	✓	✓	Custom API
Pricing model	Per token	Per token	Per second (GPU)



These three providers are complementary, not competing — the best production systems use all three based on task requirements.



Choosing the Right Provider

If your need is...	Choose
Maximum speed for real-time / voice	⚡ Groq
Fine-tuning on custom data	👉 Together AI
Image, video, audio, or 3D generation	🚀 Replicate
Largest open-source model (405B)	👉 Together AI
Fastest path from idea to demo	🚀 Replicate
Agentic pipeline (many sequential calls)	⚡ Groq
Deploy custom fine-tuned model	🚀 Replicate or 👉 Together AI
Embeddings + LLMs from one provider	👉 Together AI
Explore 10,000+ community models	🚀 Replicate

 **Production Recommendation** Use Groq for latency-critical calls. Use Together AI for fine-tuning or the 405B model. Use Replicate for multi-modal pipelines and custom model hosting. In mature architectures, all three serve different roles simultaneously.

Section Summary

The background is a solid purple color. It features a decorative pattern of small, light purple dots arranged in a grid. Overlaid on this is a white line graph with several circular nodes connected by straight lines, forming a jagged path across the center of the image.

Section Summary



Groq: Custom LPU hardware delivering 800–1,200 tokens/second — the fastest public LLM inference. ~15 curated models, OpenAI-compatible API. The clear choice when latency is the primary constraint.



Together AI: The full-stack platform. 100+ models, fine-tuning, image generation, vision models, embeddings, and LLaMA 3.1 405B access. Best for model variety and domain-specific training.



Replicate: The App Store for AI models. 10,000+ models across every modality. Zero infrastructure, per-second billing, and custom deployment via Cog. Best for multi-modal and rapid prototyping.



The Bottom Line: Speed → Groq. Fine-tuning or largest model → Together AI. Multi-modal or custom models → Replicate. These three are complementary — the best systems use all three based on task.

Hugging Face



Inference API & Transformers Library

Week 3 • Module 1: Foundations & LLM Essentials • Section 3.2

Nunnari Academy

Generative AI & Agentic AI Program

Key Topics



Hugging Face Hub

500,000+ models,
100,000+ datasets,
the GitHub of ML



Inference API

Run any Hub model
via REST — no hardware,
no setup required



Transformers Library

Unified Python interface
for loading and running
models locally



What is Hugging Face?

The central hub of the open-source AI community — often called the 'GitHub of Machine Learning.' The platform where researchers, companies, and developers share, discover, and deploy ML models.

Component	What It Is
Model Hub	500,000+ public models — world's largest open-source AI repository
Datasets Hub	100,000+ datasets ready for training and evaluation
Spaces	Free hosting for AI demos built with Gradio or Streamlit
Inference API	Hosted inference — run any Hub model via a simple REST API
Transformers	Most widely used open-source library for working with LLMs locally
Inference Endpoints	Dedicated, production-grade model deployment with private VPC



The Canonical Source

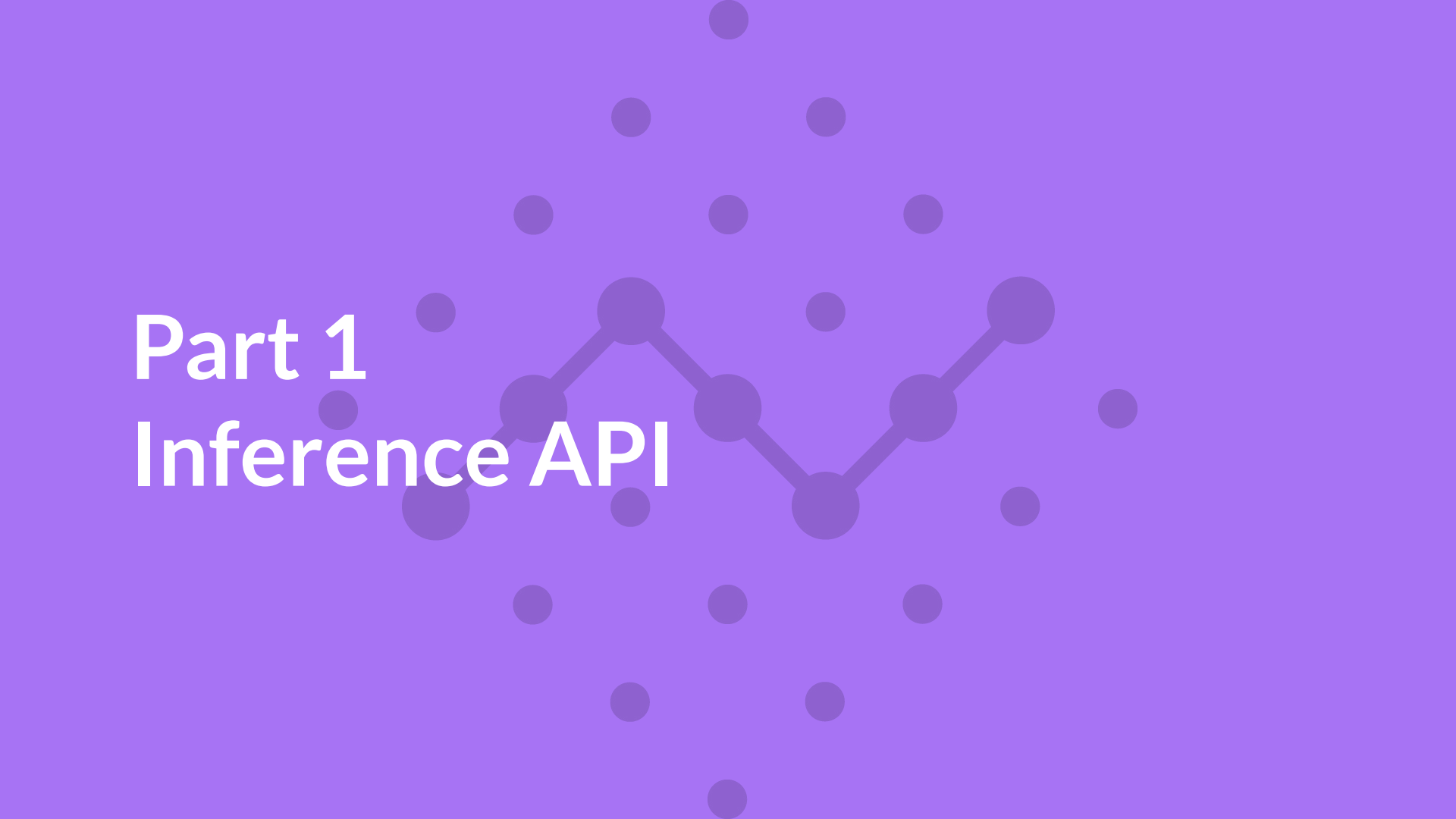
When Meta releases LLaMA, when Mistral releases a new model, when a research team publishes a fine-tuned variant — it goes to Hugging Face first. The Hub is the canonical source for open-source model weights.

HF vs Other Providers

Model repository: **500K+ (largest)**

Run locally: **✓ Transformers**

Community: **✓ Largest ML community**

The background is a solid purple color. It features a grid of small, semi-transparent purple dots. Overlaid on this grid is a line graph with several circular nodes connected by straight lines. The graph starts on the left, rises to a peak, falls to a trough, and then rises again towards the right. The text 'Part 1' is written in a white, sans-serif font on the left side of the image.

Part 1

Inference API



Inference API — Run Any Model via REST

Run any model from the Hub via a simple HTTP request — without downloading weights, without GPU hardware, and without any local setup.

Serverless API — Key Characteristics

Coverage — Supports most public models on the Hub

Cold starts — Models may take 20–60s to load if recently unused

Rate limits — Generous free tier; pay-per-use beyond that

Authentication — Simple Bearer token — your HF access token

Task detection — Automatically detects task type from model metadata

Response format — Task-specific JSON — varies by pipeline type

 **Fastest Way to Test a Model** Validate that a model works for your task in under 5 minutes using the API — before committing to local setup or fine-tuning.

Basic Usage

```
import requests

API_URL = 'https://api-inference
.huggingface.co/models/
mistralai/Mistral-7B-Instruct'

headers = {
    'Authorization':
        'Bearer hf_your_token'
}

response = requests.post(
    API_URL, headers=headers,
    json={'inputs':
        'Explain transformers.'}
)
```



Supported Tasks & Endpoints Comparison

One API, Every Pipeline

Text generation LLaMA, Mistral, GPT-style

Text classification Sentiment, topic labeling

NER (Token class) Entity extraction

Question answering Extractive QA over context

Summarization Abstractive summaries

Translation Language pair translation

Text-to-image Stable Diffusion, FLUX

Image classification ViT, ResNet

Speech recognition Whisper, wav2vec2

Embeddings Feature extraction vectors

Serverless vs Endpoints

Feature	Serverless	Endpoints
Cold starts	20-60s	None (always warm)
Rate limits	Shared	Dedicated
Privacy	Shared infra	Private VPC
Custom models	Hub public only	Any model
Pricing	Pay-per-use	Per hour
SLA	Best effort	99.9%
Auto-scaling	✗	✓ Scale to zero

⚠ Cold Starts in Production

For latency-sensitive apps, use Inference Endpoints (dedicated) which are always warm.

The background is a solid purple color. It features a decorative pattern of dots. A central path of larger dots is connected by lines, forming a zig-zag shape that passes behind the text. Other smaller dots are scattered across the background in a grid-like pattern.

Part 2

Transformers Library



Transformers Library – Core Components

The most widely used open-source library for pre-trained language models. One consistent API for hundreds of architectures – BERT, GPT-2, LLaMA, Mistral, Whisper, Stable Diffusion.

Core Components

Component	Purpose
AutoModel	Auto-loads right architecture from any Hub model ID
AutoTokenizer	Loads matching tokenizer for any model
pipeline()	High-level task interface – fastest path to inference
Trainer	Full training loop with logging, eval, checkpointing
PreTrainedModel	Base class – access raw weights and forward pass
GenerationConfig	Controls generation: temperature, top-p, beam search

pipeline() – Quick Start

```
from transformers import pipeline

# Text generation
gen = pipeline('text-generation',
               model='microsoft/Phi-3-mini')
out = gen('The key to transformers
          is', max_new_tokens=100)

# Sentiment analysis
clf = pipeline('sentiment-analysis')
r = clf('This course is great!')
# [{'label': 'POSITIVE',
#   'score': 0.9998}]

# Named entity recognition
ner = pipeline('ner',
               aggregation_strategy='simple')
ner('Nunnari Academy is in
    Coimbatore, Tamil Nadu.')
```

Nunnari Academy



Pipeline Tasks & Auto Classes

Pipeline Task Strings

"text-generation" GPT-style autoregressive

"text2text-generation" T5, BART seq2seq

"sentiment-analysis" Text classification

"ner" Named entity recognition

"summarization" Abstractive summarization

"translation_en_to_fr" Language pair translation

"question-answering" Extractive QA

"fill-mask" BERT-style token prediction

"feature-extraction" Raw embedding vectors

"automatic-speech-recognition" Whisper, wav2vec2

"image-classification" ViT classifiers

"text-to-image" Diffusion models

"zero-shot-classification" No task-specific training

Transformers

Auto Classes — Right One for the Job

`AutoModelForCausalLM`

Text gen — GPT, LLaMA, Mistral

`AutoModelForSeq2SeqLM`

Translation, summarization — T5

`AutoModelForSequenceClassification`

Text classification, sentiment

`AutoModelForTokenClassification`

NER, POS tagging

`AutoModelForQuestionAnswering`

Extractive QA

`AutoModel`

Feature extraction, embeddings



AutoModel & Generation Parameters

AutoModel + AutoTokenizer

```
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM
)
import torch

model_id = 'mistralai/Mistral-7B'
tokenizer = AutoTokenizer
    .from_pretrained(model_id)
model = AutoModelForCausalLM
    .from_pretrained(
        model_id,
        torch_dtype=torch.float16
    )

inputs = tokenizer('What is RAG?', return_tensors='pt')
outputs = model.generate(**inputs, max_new_tokens=200,
    temperature=0.7)
decoded = tokenizer.decode(outputs[0], skip_special_tokens=True)
```

Text Generation Parameters

Parameter	Controls	Range
max_new_tokens	Max tokens to generate	50-4096
temperature	Randomness	0.1→1.0
top_p	Nucleus sampling	0.85-0.95
top_k	Top-k limit	10-50
repetition_penalty	Penalize repeats	1.0-1.3
do_sample	Sampling vs greedy	True/False
num_beams	Beam search width	1-5



The Transformers library exposes every generation parameter that API providers abstract away — giving you full control over output behavior.



Quantization — Large Models on Limited Hardware

Quantization reduces memory footprint by representing weights in lower precision, making models accessible on consumer hardware.

Quantization Levels

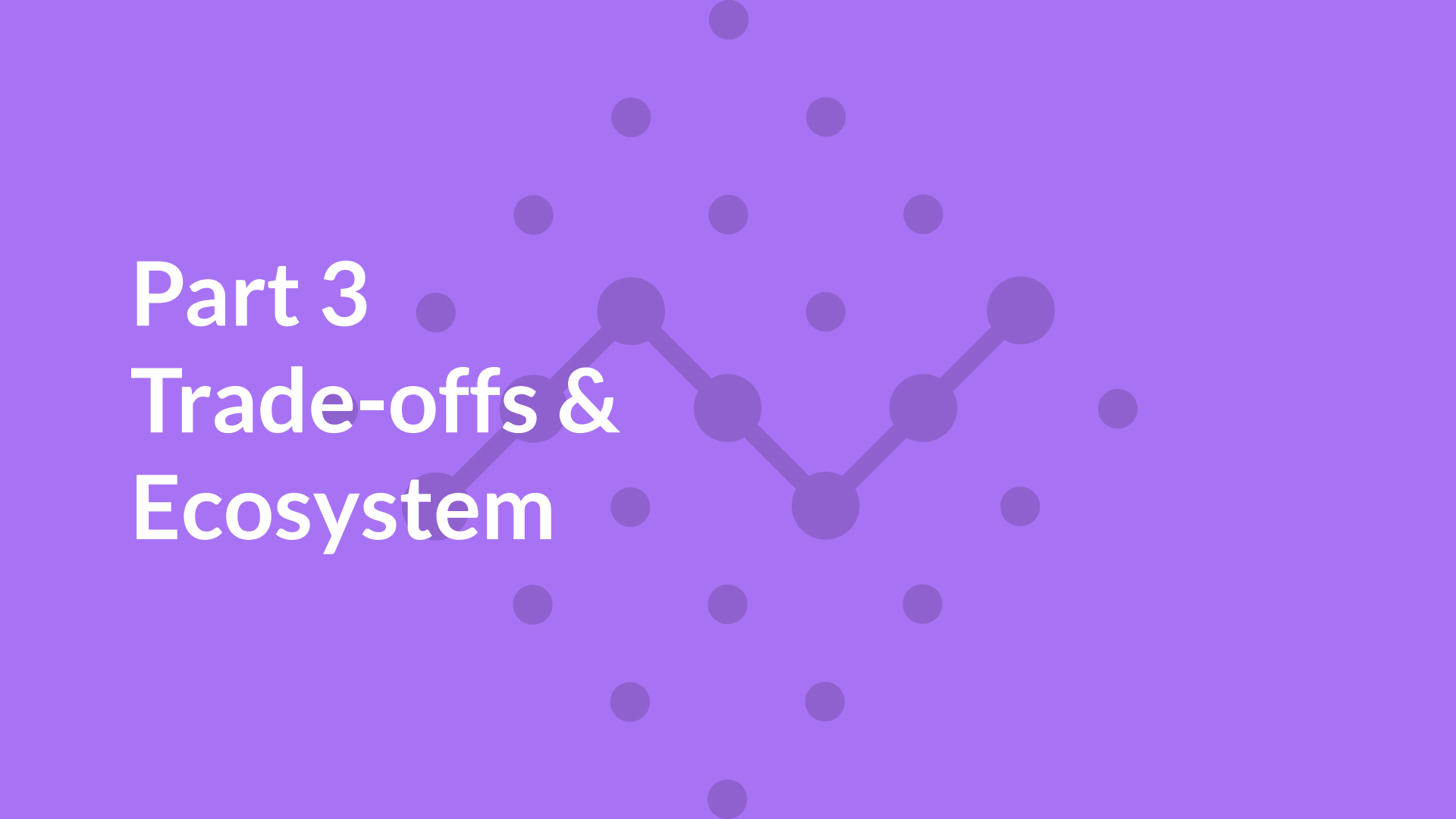
Quantization	VRAM Reduction	Quality Impact
FP32 (full)	Baseline	None
FP16 / BF16	~50%	Negligible
INT8	~75%	Minimal
INT4 (QLoRA)	~87%	Small for most tasks

Minimum VRAM by Model Size

Model	FP16	INT4
7B params	~14 GB	~4 GB
13B params	~26 GB	~7 GB
70B params	~140 GB	~35 GB
405B params	~810 GB	~200 GB



Consumer GPU Feasibility INT4 quantization makes 7B models runnable on a single consumer GPU (RTX 3060/4060 with 8–12GB VRAM). Quality degradation is minimal — the trade-off is almost always worth it for local deployment.



Part 3

Trade-offs & Ecosystem



Inference API vs Transformers – Trade-off

Dimension	Inference API	Transformers (Local)
Setup time	Minutes	Hours (GPU setup)
Hardware	None needed	GPU required
Cold starts	20–60s (serverless)	Model load once
Model selection	All Hub models	All Hub models
Data privacy	Data goes to HF	✓ Full control
Customization	Limited params	✓ Full control
Fine-tuning	✗ Not supported	✓ PEFT / TRL
Offline usage	✗ Requires internet	✓ Fully offline
Cost	Pay-per-use/free tier	Hardware cost only



Decision Framework & Ecosystem

When to Use Each

If your need is...	Use
Quickly test a model (<5 min)	 Inference API
Production pipeline with SLA	 Endpoints
Data cannot leave your servers	 Transformers
Fine-tune on custom data	 Transformers
Limited hardware / no GPU	 Inference API
500K+ models without setup	 Inference API

The HF Ecosystem

PEFT LoRA, QLoRA, adapters

TRL RLHF, DPO, SFT training

Accelerate Distributed multi-GPU training

Datasets Efficient data loading & streaming

Evaluate BLEU, ROUGE, accuracy, F1

Diffusers Stable Diffusion, FLUX

Tokenizers Fast Rust-based tokenization

Gradio Build AI demos in minutes



Coming Up in Module 2 PEFT and TRL are the standard libraries for LLM fine-tuning. When Module 2 covers QLoRA fine-tuning, these are the exact libraries you will use. Learning the Hugging Face ecosystem now pays dividends throughout the program.

Section Summary

The background is a solid purple color. It features a decorative pattern of small, light purple dots arranged in a grid. Overlaid on this is a white line graph with several circular nodes connected by straight lines, forming a jagged path across the center of the image.

Section Summary



Hugging Face Hub: The central repository of open-source AI — 500,000+ models, 100,000+ datasets. The canonical source for model weights. Every major open-source release lands here first.



Inference API: Run any Hub model via REST — no hardware, no setup. Serverless for prototyping (cold starts apply). Inference Endpoints for production — dedicated, private, SLA-backed.



Transformers Library: Unified Python interface for loading and running models locally. `pipeline()` for quick experiments, `AutoModel` + `AutoTokenizer` for full control. Quantization (INT4) makes large models accessible on consumer GPUs.



The Ecosystem: PEFT and TRL for fine-tuning, Accelerate for distributed training, Datasets for data management, Diffusers for image generation, Gradio for demos. The complete ML lifecycle in one ecosystem.

Ollama



Local LLM Deployment

Week 3 • Module 1: Foundations & LLM Essentials • Section 3.2

Nunnari Academy

Generative AI & Agentic AI Program

Key Topics



What is Ollama?

Docker for LLMs —
download, quantize &
serve with one command



REST API & Integration

OpenAI-compatible
local server on
localhost:11434



Modelfile & Customization

Dockerfile for AI —
custom prompts,
params & configs



What is Ollama?

An open-source tool that makes running LLMs locally as simple as a single terminal command. Handles downloading, quantization, hardware detection, memory management, and serving.

What Ollama Handles Automatically

Complexity	What Ollama Does
Model downloading	<code>ollama pull llama3.2</code> — downloads & caches from library
Quantization	Auto-selects best quantization for your hardware
Hardware detection	Detects GPU, offloads layers accordingly
Memory mgmt	Keeps models loaded, evicts when idle
API serving	Local REST server on localhost:11434 automatically
Model versioning	Tracks versions via Modelfile system



The Best Analogy

Think of Ollama as Docker for LLMs. Just as Docker abstracts containerizing apps, Ollama abstracts running AI models locally. Zero MLOps experience needed — run a 7B model in under 5 minutes.

Architecture

Your Application Python / JS / curl / HTTP

Ollama Server Model mgmt · API routing · GPU orchestration

llama.cpp Engine Quantized inference · GGUF · CPU+GPU hybrid

VS

Why Local? + Installation

Local (Ollama) vs Cloud APIs

Dimension	Local (Ollama)	Cloud APIs
Data privacy	✓ 100% local	Data sent to provider
Cost at scale	✓ Free (hardware only)	Per-token pricing
Internet	✓ Not required	Required always
Rate limits	✓ None	Enforced
Setup	5 min (install + pull)	✓ 2 min (API key)
Model quality	Open-source tier	✓ GPT-4o / Claude



Local Is Not a Compromise Healthcare records, legal docs, financial data, and proprietary code should often never leave your infrastructure. Ollama makes privacy-first AI practical.

Installation — Under 2 Minutes

macOS

```
brew install ollama
```

Linux

```
curl -fsSL ollama.com/install.sh | sh
```

Windows

Download installer from ollama.com

Docker

```
docker run -d -p 11434:11434 ollama/ollama
```



Zero Configuration

After install, the server starts automatically on localhost:11434. No config files, no env vars, no service management.



The Ollama Model Library

Curated library at ollama.com/library – pull any model in any size with a single command. Tag pattern: `model:size` (e.g. `llama3.1:8b`, `llama3.1:70b`).

Model	Sizes	Best For
llama3.2	1B, 3B	Lightweight tasks, edge
llama3.1	8B, 70B, 405B	General purpose, highest quality
mistral	7B	Fast general reasoning
mixtral	8x7B, 8x22B	MoE architecture, high quality
qwen2.5	7B, 14B, 72B	Multilingual, coding
deepseek-r1	1.5B, 7B, 14B, 70B	Reasoning, math, chain-of-thought
phi4	14B	Microsoft's compact powerhouse
gemma3	1B, 4B, 12B, 27B	Google's efficient models
codellama	7B, 13B, 34B	Code generation & completion
nomic-embed-text	137M	Embeddings for RAG pipelines
llava	7B, 13B, 34B	Vision + language (multimodal)



Quantization & Hardware Requirements

Quantization Levels

Tag	Memory	Quality
q2_K	Smallest	Noticeable loss
q4_0	~4 GB (7B)	Good
q4_K_M ★	~4.5 GB (7B)	Best trade-off ★
q5_K_M	~5.5 GB (7B)	Very good
q6_K	~6.5 GB (7B)	Near-original
q8_0	~8 GB (7B)	Minimal loss



q4_K_M Is the Sweet Spot Reduces 7B from ~14GB to ~4.5GB with minimal quality loss. Ollama's default choice.

Hardware Requirements

Model Size	Min RAM	Recommended
1B-3B	4 GB	Any laptop
7B-8B	8 GB	8GB+ GPU or M1/M2
13B	16 GB	12GB+ GPU or M2 Pro
30B-34B	32 GB	24GB GPU or M3 Max
70B	64 GB	Multi-GPU or M3 Ultra
405B	256 GB+	Server-grade hardware



Partial GPU Offload If model > VRAM, Ollama auto-splits across GPU + CPU for partial acceleration.



Part 1

Core CLI

Commands



Core CLI Commands

Model Management

`ollama pull llama3.2`
Download model from library

`ollama list`
List all local models + sizes

`ollama show llama3.2`
Show model details & params

`ollama rm llama3.2`
Remove model, free disk space

`ollama cp llama3.2 my-model`
Create a named model copy

Running Models

`ollama run llama3.2`
Interactive chat session

`ollama run llama3.2 "prompt"`
Single-shot prompt

`ollama run llama3.2 < file.txt`
Pipe file as input

`ollama serve`
Start server manually

Inside an Interactive Session

`/set system "You are..."` Set/change system prompt

`/show info` Display model details

`/clear` Clear conversation history

`Ctrl+D` or `/bye` Exit the session

The background is a solid purple color. Overlaid on this are several decorative elements: a network of small, semi-transparent purple dots scattered across the frame; a line graph consisting of larger purple dots connected by solid purple lines, forming a jagged path; and a faint, larger-scale grid of dots.

Part 2

The Ollama REST API



REST API – OpenAI-Compatible

Ollama serves an OpenAI-compatible REST API on localhost:11434. Any app built for the OpenAI SDK works by changing only the base URL.

Key Endpoints

Endpoint	Purpose
/api/generate	Single-turn text generation
/api/chat	Multi-turn conversation (messages array)
/api/embeddings	Generate embedding vectors
/api/tags	List available models
/api/pull	Download model from library
/api/show	Show model information

OpenAI SDK – Drop-in

```
from openai import OpenAI

client = OpenAI(
    base_url=
        'http://localhost:11434/v1',
    api_key='ollama')

resp = client.chat.completions
    .create(model='llama3.2',
    messages=[{'role': 'user',
        'content': 'Explain RAG.'}])
print(resp.choices[0]
    .message.content)
```



The Key Insight Build against the OpenAI SDK → test locally with Ollama for free → switch to OpenAI/Claude in production by changing one line (base_url). Zero code changes.



Direct REST, Streaming & Embeddings

Direct REST (no SDK)

```
import requests, json
resp = requests.post(
    'http://localhost:11434/api/chat',
    json={'model':'llama3.2',
          'messages':[{'role':'user',
                        'content':'What is RAG?'}],
          'stream': False})
print(resp.json()['message']['content'])
```

Generating Embeddings

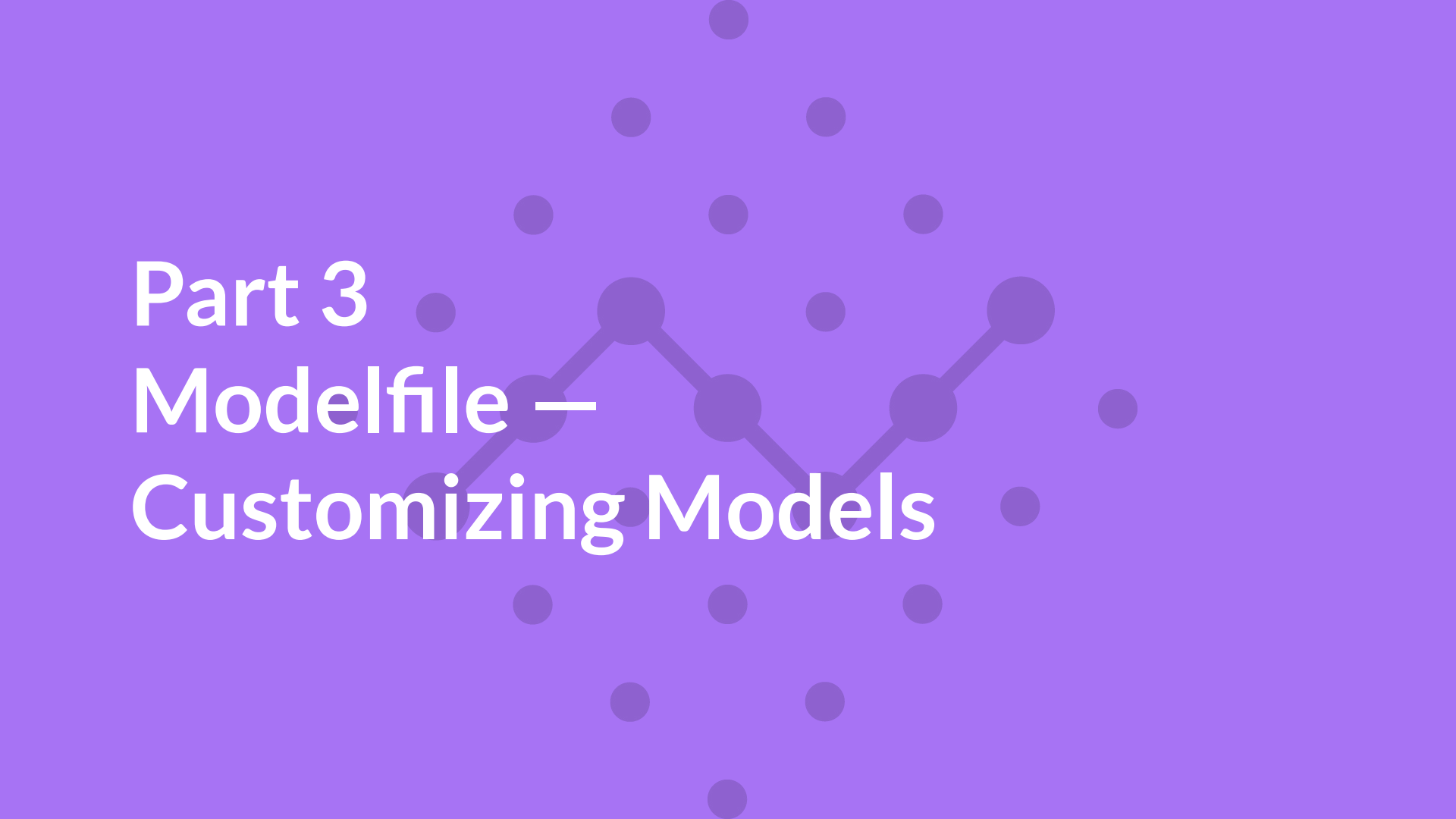
```
resp = requests.post(
    'http://localhost:11434/api/embeddings',
    json={'model':'nomic-embed-text',
          'prompt':'Nunnari Academy teaches GenAI'})
emb = resp.json()['embedding']
# 768-dim float vector
```

Streaming Responses

```
# stream: True for real-time tokens
resp = requests.post(url, json={
    ..., 'stream': True}, stream=True)
for line in resp.iter_lines():
    chunk = json.loads(line)
    print(chunk['message']['content'],
          end='', flush=True)
```

Multi-Turn Conversation

```
history = []
def chat(msg):
    history.append({'role':'user','content':msg})
    resp = requests.post(url, json={
        'model':'llama3.2',
        'messages':history, 'stream':False})
    reply = resp.json()['message']['content']
    history.append(
        {'role':'assistant','content':reply})
    return reply
```



Part 3

Modelfile —

Customizing Models



Modelfile – Dockerfile for AI

A text file defining a customized model variant – system prompt, generation parameters, and behavior. Build once, share with your team, run anywhere.

Modelfile Syntax

```
FROM llama3.2

SYSTEM """
You are TamilBot, a helpful AI
assistant for Nunnari Academy.
Respond in Tamil when the user
writes in Tamil.
"""

PARAMETER temperature 0.7
PARAMETER top_p 0.9
PARAMETER num_ctx 8192
PARAMETER num_predict 512

# Build & run custom model
ollama create tamilbot -f Modelfile
ollama run tamilbot
```

Modelfile Parameters

Parameter	Controls	Example
FROM	Base model	llama3.2
SYSTEM	System prompt	"You are a tutor..."
temperature	Randomness	0.7
top_p	Nucleus sampling	0.9
num_ctx	Context window	8192, 32768
num_predict	Max gen tokens	512, 2048
stop	Stop sequences	"< eot_id >"
TEMPLATE	Chat template	Custom format

Use Cases

- Role-specific assistants (support, tutoring, code)
- Task-tuned params (deterministic for code, creative for writing)
- Share configs via version control



Part 4-5

GPU Acceleration & Network Serving



GPU Acceleration & Network Serving

Supported Hardware

NVIDIA GPU — Full CUDA — best performance, 6GB+ VRAM

Apple Silicon (M1-M4) — Native Metal — unified memory, excellent

AMD GPU — ROCm support on Linux

CPU only — Always works, slower (5-15 tok/s)

Partial GPU offload — Auto-splits model across GPU + CPU

Performance — 7B (q4_K_M)

RTX 4090 (24GB) ~80-120 tok/s

RTX 3060 (12GB) ~30-50 tok/s

Apple M3 Pro (18GB) ~25-40 tok/s

CPU only (modern) ~5-15 tok/s

Serving on a Network

Variable	Effect
OLLAMA_HOST	Bind to 0.0.0.0 for network
OLLAMA_ORIGINS	Allow cross-origin requests
OLLAMA_MODELS	Custom model storage path
OLLAMA_NUM_PARALLEL	Concurrent requests
OLLAMA_MAX_LOADED	Max models in memory

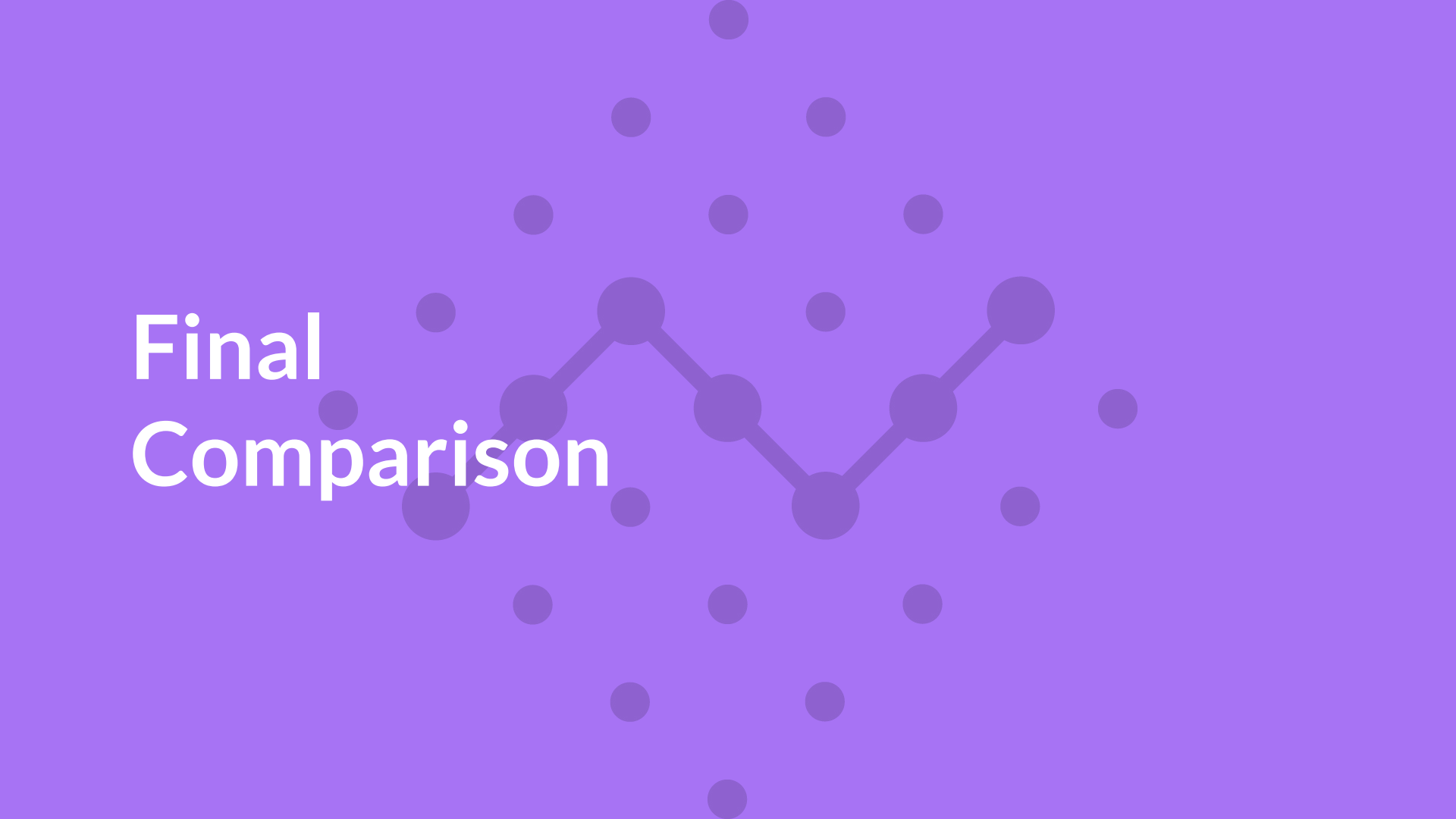
```
# Enable network access
OLLAMA_HOST=0.0.0.0:11434 \
ollama serve
# Team: http://192.168.1.50:11434
```



Security Warning

Never expose to public internet without auth. Use private LAN or Tailscale VPN. Place nginx with basic auth for public access.

Final Comparison

The background is a solid purple color. Overlaid on this is a network of dots and lines. Most dots are small and light purple, arranged in a grid-like pattern. A specific path of seven dots is highlighted with larger, darker purple dots, connected by thick, dark purple lines. This path starts on the left, moves up-right, then down-right, then up-right, then down-right, then up-right, and finally down-right, forming a zig-zag shape across the center of the image.



Ollama vs Cloud APIs vs HF Transformers

Dimension	Ollama	Cloud APIs	HF Transformers
Setup time	5 min	2 min	Hours
Data privacy	✓ Full	Sent to provider	✓ Full
Internet	✗ Not needed	Required	✗ Not needed
Technical skill	Beginner-friendly	Easy	ML knowledge
OpenAI compat	✓ Drop-in	✓ Native	✗ Own API
Fine-tuning	✗ Not supported	Limited	✓ PEFT / TRL
Embeddings	✓ nomic-embed	✓ Provider APIs	✓ Any model
Cost model	Free (HW only)	Per-token	Free (HW only)
Best for	Privacy, offline, dev	Max quality, scale	Research, fine-tune



When to Use Ollama

Choose Ollama When

- ✓ Data privacy is critical — healthcare, legal, proprietary code
- ✓ Offline capability — edge, air-gapped, unreliable internet
- ✓ Cost optimization at scale — fixed HW cost beats per-token
- ✓ Development & testing — build locally before deploying to cloud
- ✓ Team experimentation — shared server, no API costs
- ✓ Learning & research — explore models freely
- ✓ Latency-sensitive local apps — no network round-trip

Not Ideal When

- ✗ Need absolute largest models — GPT-4o / Claude Opus quality not yet open-source
- ✗ Hardware too limited — even quantized small models need 4GB+ RAM
- ✗ Need fine-tuning capability — use HF Transformers + PEFT/TRL instead
- ✗ Need maximum inference speed — Groq's LPU is faster than consumer GPU

Section Summary

The background is a solid purple color. It features a decorative pattern of small, light purple dots arranged in a grid. Overlaid on this is a white line graph with several circular nodes connected by straight lines, forming a jagged path across the center of the image.

Section Summary



Ollama: Docker for LLMs. Handles downloading, quantization, hardware detection, memory management, and REST serving automatically. Install in 2 minutes, run any model with one command. Built on llama.cpp + GGUF.



How It Works: Serves OpenAI-compatible REST API on localhost:11434. Auto-uses GPU with partial offload for oversized models. q4_K_M quantization is the default sweet spot — 7B models fit in ~4.5GB.



API Usage: Use OpenAI SDK with base_url='http://localhost:11434/v1' for drop-in compatibility. Or call /api/chat and /api/generate directly. Stream tokens in real-time. Generate embeddings with nomic-embed-text.



Modelfile: Customize any model with system prompt, generation parameters, and behavior — equivalent to a Dockerfile for AI. Build once, share as text file, run anywhere with ollama create.



Use Case: The right choice when data privacy, offline capability, or cost at scale matter. Not a replacement for cloud APIs when maximum model quality or fine-tuning is needed.

Thank You!



Nunnari Academy

Generative AI & Agentic AI Program

Week 3 • Module 1: Foundations & LLM Essentials